



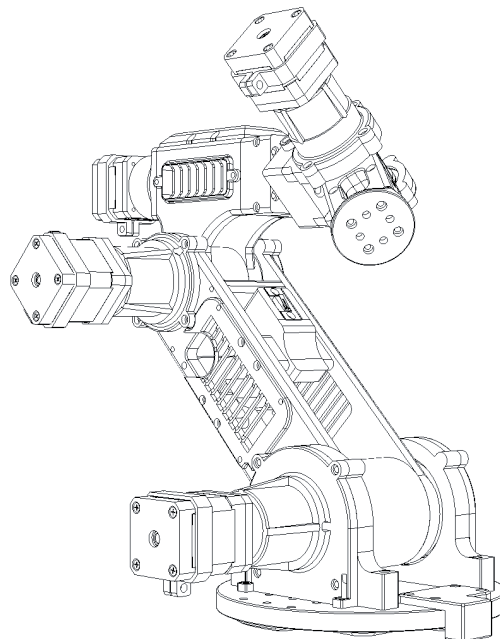
Degree Project in Technology

First cycle, 15 credits

Open Source Five-axis Robotic-arm

Design, Construction and Evaluation

SEBASTIAN VALLIN



Open Source Five-axis Robotic-arm

Design, Construction and Evaluation

SEBASTIAN VALLIN

Degree Programme in Electronics and Computer Engineering

Date: June 18, 2025

Supervisor: Carl-Mikael Zetterling

Examiner: Ana Rusu

School of Electrical Engineering and Computer Science

Swedish title: Fem-axlad Robot-arm med öppen källkod

Swedish subtitle: Design, Konstruktion och Utvärdering

Abstract

This thesis examines the repeatability and precision achievable with a self-built, five-axis robotic arm developed using standardized hardware and 3D-printed components on a limited budget. The project is fully open-source, and all design decisions were made with accessibility and reproducibility in mind. The robotic arm builds on an early prototype from prior work and has been further refined and evaluated. The goal is to provide a documented, tested, and open-source robotic arm for builders to engage with and continue to develop. Evaluation was conducted using custom low-resource test methods to assess accuracy, repeatability, deflection, backlash, load capacity, and reach. Results indicate mixed performance and highlight several areas for further improvement. All files for 3D-models, printed circuit board (PCB), and software are available in the project's GitHub repository.

Keywords

3D-printing, Design process, Computer-aided design, Joints, Stepper motor, Stepper driver, Micro-stepping, Gear reduction, Torque, Speed, 3D-model, Assembly, Open-source, End-stop, Accuracy, Repeatability, Load capacity, Deflection, Backlash, Reach, Mathematical model, Forward kinematics, Inverse kinematics

Sammanfattning

Denna avhandling utforskar upprepbarhet och precision som är möjlig med en egen-byggd fem-axlad robot-arm. Robot-armen utvecklades under begränsad budget med standardiserad hårdvara och 3D-utskrivna delar. Projektet har öppen källkod och all design har skett med hänsyn till tillgänglighet och återskapande. Robot-armen bygger på en tidigare-utvecklad prototyp som har förbättrats och utvärderats. Målet med robot-armen är att erbjuda en dokumenterad, testad robot-arm med öppen-källkod för att accelerera dess fortsatta utveckling. Utvärdering utfördes med själv-utvecklade test-metoder för att säkerställa precision, upprepbarhet, deflektion, glapp, arbetsområde samt belastningskapacitet. Erhållna resultat visar blandad prestanda med möjligheter till förbättring. Alla filer för 3D-modeller, kretskorts samt mjukvara finns tillgängligt i projektets GitHub arkiv.

Nyckelord

3D-Printing, Designprocess, Dator hjälpt design, Leder, Stegmotor, Steg-motor drivare, Microsteg, Nedväxling, Vridmoment, Hastighet, 3D-Model, Montage, Öppen-källkod, Ändstopp, Precision, Upprepbarhet, Glapp, Deflektion, Arbetsområde och Belastningskapacitet, Matematisk modell, Framåt kinematik, Inverterad kinematik

Acknowledgments

I would like to express my sincere gratitude to Carl-Mikael Zetterling for hosting the excellent course IE1332. The course inspired me to delve deeper into PCB design and electrical components, enabling me to develop several PCBs for my own projects. Without the knowledge gained from the course, the drink mixer and robot arm projects would not have been possible. Thank you.

I would also like to thank Voron Design for their hard work on open-source 3D printer designs. The multiple Voron 3D printers I built helped me understand the fundamentals of mechatronics, hardware, computer-aided design, and assembly techniques—all of which were utilized in this degree project. Thank you.

Stockholm, June 2025
Sebastian Vallin

Contents

1	Introduction	1
1.1	Background	2
1.2	Problem	2
1.3	Project goal	3
1.4	Purpose	3
1.5	Sustainability	3
1.6	Methods	4
1.7	Stakeholders	5
1.8	Delimitations	5
1.9	Structure of the thesis	6
2	Background	7
2.1	Prior work on the robotic arm	7
2.1.1	Robotic-arm prototype	7
2.1.2	Issues discovered	10
2.2	Experience	10
2.2.1	Voron-related contributions	11
2.3	Drink-mixer project	11
2.4	PCB development	12
2.5	Summary	12
3	Methodology	13
3.1	Design inspiration	13
3.2	Designing for open-source	14
3.2.1	Print-ability	15
3.3	3D-Printing	15
3.3.1	Choosing material	15
3.3.2	Engineering-grade materials	16
3.3.3	Slicer parameters	16

3.3.4	Required amount of plastic filament	17
3.3.5	3D-printers used in the project	17
3.4	Gear reductions	18
3.4.1	Belted gearboxes	19
3.4.2	Cycloidal gearboxes	20
3.4.3	Planetary gearboxes	20
3.5	Building a mathematical model	21
3.5.1	Measuring the robotic-arm	21
3.5.2	Forward kinematics	22
3.5.3	Inverse kinematics	24
3.6	Testing	24
3.6.1	Repeatability test	24
3.6.2	Load capacity test	26
3.6.3	Accuracy test	27
3.6.4	Deflection test	28
3.6.5	Backlash test	28
3.6.6	Reach simulation	29
4	Design and Construction	31
4.1	Design	31
4.1.1	Issues encountered	32
4.2	Mechanics	34
4.2.1	Joint assemblies	34
4.3	Electronics	39
4.3.1	Overview	39
4.3.2	Controller and PCB-shield	40
4.3.2.1	Tinkerer 1.0 overview	40
4.3.3	Stepper Motors	42
4.3.3.1	Stepper motors and feedback-loops	42
4.3.3.2	Stepper motor parameters	43
4.3.3.3	Stepper drivers	43
4.3.3.4	Amperage tuning and micro-stepping	43
4.3.3.5	Stepper driver TMC2226	44
4.3.3.6	Stepper driver DRV8825	44
4.3.4	Choosing power supply	45
4.3.5	End-stops	45
4.3.6	Custom wire-loom	46
4.4	Software	48
4.4.1	Programming language	48

4.4.2	Interfacing with the Raspberry	48
4.4.3	PCB Shield pinout	49
4.4.4	Main program	49
4.4.5	Python library - pigpio	49
4.4.5.1	Movement functions	50
5	Results and Evaluation	53
5.1	Results	53
5.2	Evaluation	59
6	Conclusion and Future work	65
6.1	Future work	66
6.1.1	Monitoring GitHub page	66
6.1.2	Designing add-on end-effectors	66
6.1.3	Tinkerer 1.0 optimizations	66
6.1.4	Nema-23 Stepper motors	67
6.1.4.1	New stepper drivers	67
6.1.5	Explore cycloidal-gearboxes	67
6.1.6	C-based software	67
6.1.6.1	Acceleration & deceleration	68
6.1.6.2	Inverse kinematics	68
	References	69
A	Tinkerer 1.0 schematics and pictures	72
A.1	Schematics	73
A.2	Gallery	74
B	Cost, BOM and Miscellaneous data	75
B.1	Bill of materials and cost	75
B.2	Build data	77
C	Tools used	78
C.1	Physical tools	78
C.2	Development software	79
D	Code	80
D.1	Pinout	80
D.2	Main-program	82
D.3	Forward Kinematics	91

E Accuracy Measurements

92

List of Figures

2.1	CAD render of a blind joint assembly. Allen-key and button-head screw highlighted in blue.	8
2.2	CAD render of complete aluminum-profile frame.	8
2.3	Non 3D-printable hardware, modeled vs real.	9
2.4	CAD render of prototype robotic arm.	9
2.5	CAD renders of drink-mixer and motion assemblies.	11
2.6	3D-view of developed PCBs.	12
3.1	Slicer examples showcasing correct vs incorrect orientation.	17
3.2	FDM 3D-printers used in the project.	18
3.3	CAD render of a 2:1 gear-reduction.	19
3.4	CAD render showcasing a belted reduction driven by a stepper motor. The 16-tooth pulley is driving an 80-tooth pulley through the help of a timing belt, resulting in a gear reduction of 5:1.	20
3.5	CAD render showcasing how lengths in table 3.1 was determined.	22
3.6	Location for end-effector in homed position.	24
3.7	CAD renders of self-made tools.	25
3.8	Setup for repeatability-tests with tools presented in fig 3.7. The needle is centered on the target plate in both figures.	26
3.9	End-effector position estimation from scaled photo.	27
3.10	Setups for deflection test. Figure 3.10a showcases the discarded tri-pod setup, and Figure 3.10b showcases the used setup for testing.	28
4.1	CAD render of project robotic-arm	32
4.2	Final design of the robotic arm. The arm is resting in its folded position.	32
4.3	Tight and loose fit of a non-shrink compensated ABS-part.	33

4.4	Torque transfer assembly with dual-bearing support and a click-wheel for end-stop.	33
4.5	CAD render showcasing all five joints and their assembly color-coded.	34
4.6	CAD render showcasing an exploded view of the first rotation joint mechanics. Highlighted parts are key for base-plate rotation (some aluminum extrusions are hidden to aid visibility).	35
4.7	CAD render showcasing an exploded view of the first arm joint mechanics. Highlighted parts are key for rotation.	36
4.8	CAD render showcasing an exploded view of second arm joint mechanics. Highlighted parts are key for rotation.	37
4.9	CAD render showcasing an exploded view of second rotation joint mechanics. Highlighted parts are key for rotation.	38
4.10	CAD render showcasing an exploded view of third arm joint mechanics. Highlighted parts are key for the rotation of the arm.	39
4.11	CAD render showcasing the electronics compartment of the robotic arm. Electronics is highlighted in blue.	40
4.12	CAD render of Tinkerer 1.0.	41
4.13	CAD render showcasing the location of the five end-stop micro-switches used for homing.	46
4.14	CAD render showcasing a correctly placed end-fitting on a wire - ready to be crimped. End-fitting is highlighted in blue.	47
4.15	Cable-loom routing.	48
5.1	Region where data-points are located from both horizontal and vertical tests.	53
5.2	Results from 10° displacement.	54
5.3	Results from 20° displacement.	54
5.4	Results from 30° displacement.	55
5.5	Results from homing displacement.	55
5.6	Vertical repeatability test results.	56
5.7	Deflection of the robotic-arm in μm under different loads.	57
5.8	End-effector reach in the XZ-plane.	58
5.9	CAD render of backlash in rotational movement.	60
5.10	CAD render of test-positions about the center of the base-plate.	60
5.11	Data-sheet of speed vs torque for the stepper motor used.	61
5.12	Reach of the robotic arm overlapped with simulated reach derived from forward-kinematics data-points.	62
A.1	Front view of Tinkerer 1.0	74

A.2	Back view of Tinkerer 1.0	74
E.1	End-effector position at: $0^\circ, 90^\circ, 90^\circ, 0^\circ, 90^\circ$	92
E.2	End-effector position at: $0^\circ, 135^\circ, 135^\circ, 0^\circ, 90^\circ$	93
E.3	End-effector position at: $0^\circ, 45^\circ, 45^\circ, 0^\circ, 45^\circ$	93
E.4	End-effector position at: $0^\circ, 90^\circ, 180^\circ, 0^\circ, 90^\circ$	94
E.5	End-effector position at: $0^\circ, 180^\circ, 180^\circ, 0^\circ, 90^\circ$	95

List of Tables

3.1	Arm joint lengths.	21
3.2	DH-parameters for robotic-arm	23
3.3	End-effector position using FK	23
4.1	Feedback-loop comparison for stepper motors	42
4.2	Absolute-positioning examples	50
4.3	Relative-positioning examples	51
5.1	Accuracy results real vs FK	58
B.1	BOM and project price breakdown	76
B.2	Miscellaneous build-data	77
C.1	Tools used in the assembly	78
C.2	Software used in the project	79

Listings

- 4.1 Example of pin-name mapping. 49
- 4.2 pigpio installation. 49
- 4.3 pigpio daemon startup. 50
- 4.4 GPIO initialization using pigpio 50
- 4.5 Sending step-signals using pigpio 51
- 4.6 End-stop GPIO reading using pigpio 52
- 4.7 Homing sequence 52

List of acronyms and abbreviations

5-DOF	Five Degrees Of Freedom
CAD	Computer Aided Design
DIY	Do It Yourself
DOF	Degrees Of Freedom
FK	Forward Kinematics
GPIO	General-Purpose Input/Output
IK	Inverse Kinematics
NC	Normally Closed
NO	Normally Open
PCB	Printed Circuit Board
PID	Proportional-Integral-Derivative
PWM	Pulse Width Modulation

Chapter 1

Introduction

The integration of 3D printing and computer-aided design (CAD) software has transformed do-it-yourself (DIY) projects. Unlike traditional DIY efforts limited by locally available materials, modern makers can now design and produce custom components with high precision. This capability supports complex, collaborative projects within open-source communities.

This thesis contributes a reproducible blueprint for a DIY 5-degree-of-freedom (DOF) robotic arm, designed and tested for the DIY community. The arm's design and construction are central, directly influencing its performance and replicability. Building on a flawed earlier prototype, the final design reuses components, such as motors, to minimize costs. Standardized hardware from various vendors and 3D-printed parts, including arms, panels, and bearing housings, were utilized. Without access to advanced tools like laser trackers, custom test setups were developed to evaluate the arm's performance and reliability. These setups were also assessed for effectiveness. While the success of a DIY build depends on the chosen hardware and assembly quality, this does not justify neglecting rigorous testing.

To distinguish this robotic arm from other DIY projects, the design emphasizes accessibility and completeness to attract new builders and accelerate collaborative development within the open-source community. Additionally, the design incorporates heat-set inserts throughout, providing flexibility for future modifications and enhancing its appeal to makers.

1.1 Background

This robotic-arm project draws inspiration from both industrial robotic arms used in assembly lines and the open-source initiative Voron Design [1], which develops high-performance 3D-printers using 3D-printed components and standardized hardware. Voron Design has pioneered reliable and repeatable methods for fastening sub-assemblies to 3D-printed parts using heat-set inserts. Their techniques have become influential enough that some heat-set insert manufacturers have acknowledged Voron Design by stamping the Voron logo on their products [2] [3]. Prior work related to Voron Design and self-developed projects, which is further detailed in chapter 2, demonstrates that complex and robust assemblies are achievable using these techniques.

The robotic-arm project unofficially started with a printed-circuit-board (PCB) design for stepper motor applications. As the PCB supports five stepper motors, the idea emerged to build a robotic arm with five degrees of freedom (5-DOF). A prototype design was made using CAD software Fusion 360, and stepper motors were ordered for the build. When assembling the first arm joint, critical flaws were discovered. The robotic-arm project quickly lost traction and was halted for a year. The robotic-arm idea was revived when a project was needed for the degree project. With positive experience of open-source projects like Voron Design, it was clear that an open-source 5-axis robotic arm would be a suitable project and a second chance at completing the robotic arm project.

1.2 Problem

With a custom-built robotic arm on a budget, the challenge is to determine its performance. Proper testing requires expensive equipment and is out of reach for most DIY initiatives or lone creators. Research must be carried out to find relevant measurable performance metrics to test, and custom, well-thought-out setups must be built to provide an adequate answer. Through testing and evaluation, the next revision of the robotic-arm could achieve better performance. The problem is further refined with the question *“How can repeatability, accuracy, load capacity, deflection, reach, and backlash of a DIY robotic arm be effectively tested using low-cost improvised methods instead of professional equipment?”*

1.3 Project goal

The goal of this project is to design, construct and test a functional open-source 5-DOF robotic arm with use of 3D-printed parts and standardized, commonly available hardware on a budget for DIY hobbyists. The evaluation is based on results from repeatability, accuracy, load capacity, deflection, backlash, and reach tests. The goal is divided into following three sub-goals:

1. Design and construct a 5-DOF robotic arm, integrating mechanical, electrical, and control systems.
2. Develop test setups to measure the arm's kinematic and structural performance.
3. Analyze test results to evaluate the arm's functionality and performance against design specifications.

1.4 Purpose

The purpose of this project is to develop a low-cost, open-source 5-DOF robotic arm that serves as an accessible platform for hobbyists and students to learn basic Mechatronics, including mechanical design, electronics, mathematical models for robotics and gearboxes. By providing a fully tested design with documented performance metrics, the project aims to enable builders to gain both practical and theoretical experience in robotics and encourage community-driven improvements through open-source contributions to the project.

1.5 Sustainability

Re-purposing unused hardware ordered for the robotic-arm prototype is sustainable by definition. 3D-printed parts ideally only use the required amount of plastic with no leftover waste. With a design that aids printability, the waste of plastic when printing required parts can be reduced. With a modular design utilizing heat-set inserts, parts of the assembly can be attached and detached without screw threads damaging the plastic part. If a printed part of the robotic arm is damaged, it can be reprinted and added back.

Since DIY builds depends on the builder and equipment, it's hard to make an assessment on environmental impact. Intuitively, it makes more sense to

order ready made robotic arms from manufacturers with optimized assembly lines. However, ready made robotic arms defeats the educational process of building the robotic arm and a viral DIY project could spark interest among vendors to offer full hardware kits for builds if deemed profitable, giving the project the potential for improved sustainability over time.

1.6 Methods

Most construction methods used in the robotic-arm project originate from past projects and experience. Notable similar work is elaborated on in chapter 2. A functional robotic arm is key to achieving goals set in section 1.3, and given a time frame of 10 weeks, the design and build were immediately started. Test methods for the robotic arm involve accuracy, load capacity, deflection, backlash, reach and repeatability tests.

Research was conducted in suitable gearboxes for robotic arms, mathematical kinematic models, tests and setups for evaluating a robotic arm and finally datasheet parameters for used hardware. Although used gearboxes were predefined through re-purposing of bought motors, it was deemed critical to analyze other gearbox solutions to determine points of improvement for the future. Mathematical kinematic models used for robotic arms are critical for motion and analysis, and forward kinematics is especially important for solving theoretical positions in space of the robotic arm. Both forward and inverse kinematics are directly related to the length and layout of joints, and is thus individual for every robotic arm. Test methods were derived through understanding the key-parameters of robotic arms. Once key-parameters were identified and understood, custom test-setups could be constructed for each parameter.

Methods used are mostly well documented and universal to other mechanical systems, and some were improvised like the accuracy test which stem from exploiting distances in a photo through scaling pictures in CAD software. Methods involving expensive equipment were discarded, although, a high-resolution dial gauge with a estimated brand-new price of 100€ – 200€ was lent. Dial gauges are common measurement equipment and can be found easily from multiple vendors. It's expected that most DIY hobbyists can get either lend or buy a used dial gauge and its usage in tests is therefore exempted from the high cost rule.

1.7 Stakeholders

With open-source projects come decisions that affect builders and, by extension, future stakeholders. The following six points are taken into account in the development of the robotic arm:

1. Use of common standardized components to ensure part availability for builders.
2. Use of low-cost components to ensure affordability for builders.
3. Development transparency through documentation and files on GitHub.
4. Modular design to ease repairability and mod-ability.
5. Unused anchor-points in the form of heat-set inserts to provide room for modifications.
6. 3D-printed parts designed with printability in mind, helping builders to print more easily.

1.8 Delimitations

Stepper motors are expensive hardware and were ordered for the build of the robotic-arm prototype. As the project is self-funded and has a limited budget, new stepper motors won't be purchased. Another limitation of the project is time. Developing a functional robotic arm is the main goal in order to carry out tests for evaluation. If the project is short on time, less important features must be cut from the project. The following features risk getting cut in favor of the core of the project:

1. End-effectors for object interactions.
2. Mathematical models for advanced movement algorithms.
3. Software code optimizations for performance and precision.
4. Test method refinements for better results.

1.9 Structure of the thesis

The chapters of the thesis are written with no expectations of prior knowledge from the reader. Detailed background to the robotic-arm project can be found in chapter 2. Robotic arms belong to an engineering field called *Mechatronics*, which is the synergy between Electrical, Computer, Mechanical, and Control-engineering [4]. Electronics-related work can be found in section 4.3 and software in section 4.4. Mechanics is found in section 4.2, and control is found inside both electronics and software sections and has no dedicated chapter. The Mechatronics sections can be read individually, but it's suggested that all three sections be read in order, as it aligns with the development time frame of the project.

Chapter chapter 3 covers the test methods used, mathematical models for robotic arms, gearbox-solutions, design-methods, and 3D-printing methods. Finally, a conclusion to the project and future work to be done can be found in chapter chapter 6. Schematics and images of the custom PCB-shield used can be found in appendix A. Price, BOM, and miscellaneous data from the build can be found in B. Tools used for the build are presented in appendix C and the complete code for the project in appendix D. Measurements used in the accuracy test can be found in E

Chapter 2

Background

This chapter describes previous projects and work that are relevant to the development of the robotic arm. As development and build rely heavily on experience from similar projects of the past, it's advised that this chapter be read first before moving into further chapters.

2.1 Prior work on the robotic arm

The robotic arm built in this thesis was based on an earlier prototype design. This section covers work done on the prototype, what was carried over, and what was discarded.

2.1.1 Robotic-arm prototype

The base of the prototype robotic arm was chosen as the starting point of the design. A square rig was constructed using aluminum profiles. Aluminum profiles are standardized hardware that comes in various sizes. To create a construction out of aluminum profiles, a technique called *blind joint assembly* is used. A screw is put into one end of profile **A**. The next step is drilling a hole in profile **B**. Then, the screw head can slide into the tracks of profile **B** and the drilled hole allows access to the screw head for tightening. Figure 2.1 below visualizes the blind joint concept.

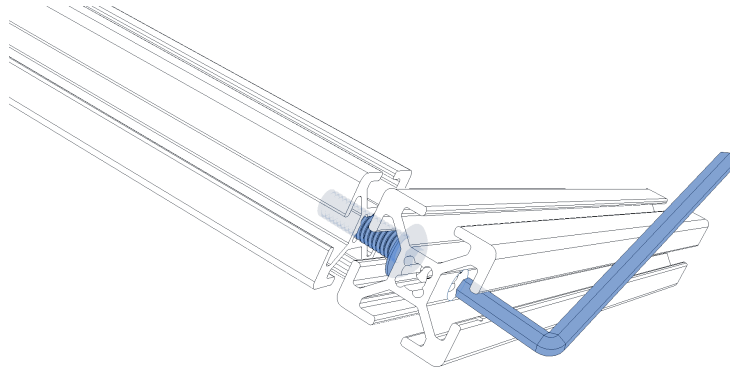


Figure 2.1: CAD render of a blind joint assembly. Allen-key and button-head screw highlighted in blue.

The hole placement is important in a blind joint assembly. A misplaced hole results in a non-square end result. To ensure that hole locations are correct, 3D-printed jigs were used. Another critical step is to ensure that the surface on which the frame is assembled is completely flat. Utilizing a sheet of glass or similar as a build surface helps. The frame construction used in the project as the base of the project robotic arm can be seen in Figure 2.2 below.

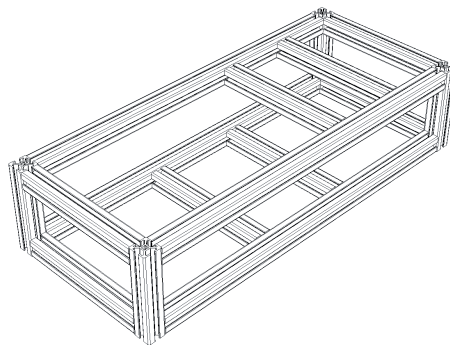


Figure 2.2: CAD render of complete aluminum-profile frame.

Some hardware does not exist in public CAD libraries. Critical components for the robotic arm that was bought had to be designed from

measurements using a caliper. This process is important as the 3D-printed parts, which interface with the hardware, rely on a good foundation. Figure 2.3 below shows models of non-3D-printable hardware used for the final project robotic arm, but the largest bearing and largest flange-coupler were used in the prototype.



(a) CAD models of hardware



(b) Real hardware

Figure 2.3: Non 3D-printable hardware, modeled vs real.

With hardware designed, the prototype robotic arm was developed. Arm joints were built using a single bearing support to enhance their looks. All 3D-printed parts were given curved surfaces and complex geometries. Figure 2.4 below pictures how far the design of the prototype came along until abandoned.

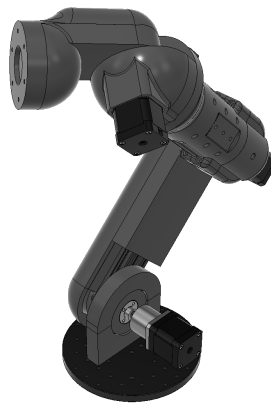


Figure 2.4: CAD render of prototype robotic arm.

2.1.2 Issues discovered

The prototype robotic arm was built with form over function, resulting in several problems. Firstly, the robotic arm was too long (70cm). Motors used in the project would not be able to support the weight with their limited permissible torque, limiting its load capacity. Secondly, the 3D-printed parts weren't print-friendly for an open-source design, reducing the number potential builders. Thirdly, repeated re-use of the same bearing model led to all arm joints having similar cross-sectional areas, which adds unnecessary weight. Lastly, the prototype joints were only supported by a single bearing with a height of 10mm. The bearings bought are of budget quality, and the prototype was destined for catastrophic accuracy with a single bearing support solution as the arm would bend sideways at each joint.

After evaluation, it was clear that the arm required a redesign. Every 3D-printed part was scrapped, and the non-3D-printed hardware was kept for a new revision.

2.2 Experience

Mentioned in section 1.1, Voron Design related work and experience are key to the project. The three 3D-printers used in the project are Voron 2.4 [5] printers and were sourced and built by me. A build takes about 40 hours to complete, and key parts of the build teach the following:

1. Assembling rigid frames using aluminum profiles
2. Gearbox reductions involving timing belts.
3. Belted pulley systems.
4. Stepper motors, stepper drivers, and related parameters.
5. Best practices when assembling 3D-printed sub-assemblies.
6. Best practices when designing 3D-printed sub-assemblies.
7. Common standardized mechanical hardware.
8. AC-to-DC power supplies and wiring involved.
9. Homing and end-stops.
10. Creation of wire-looms from scratch.

11. Supporting wire-loom correctly in moving systems.

2.2.1 Voron-related contributions

Two out of my three Voron printers are based on my base assembly design [6], which allows for a build size that is officially not supported. Voron printers can be built in several sizes, and supported sizes are $250mm^3$, $300mm^3$, and $350mm^3$. The modification allows for $150mm^3$ size, which shrinks belt lengths in belt-driven systems, allowing for reduced weight and resonances, resulting in improved print speeds and print quality.

Another contribution is to the Voron Switchwire 3D-printer, where an external enclosure was designed. Panels of the enclosure are fastened with magnetic clips and are my most successful project on GitHub to date [7].

The Micron 3D-printer is a Voron-inspired derivative where Voron 2.4 is built using shrunken hardware and re-designed parts to fit. Contributions to this project involve a mounting solution for print-head PCB, an enclosure-lighting solution, and a gear-box remix to fit aluminum-based pulleys instead of 3D-printed pulleys [8].

2.3 Drink-mixer project

In an engineering project course, my project team decided to build a drink mixer. I was given the role of product architect and designed, produced, and built two motion-test assemblies and the drink mixer itself within a four-week time frame. These achievements were made possible with the acquired knowledge from the Voron builds and contributions. Figure 2.5 below showcases the three complete assemblies.

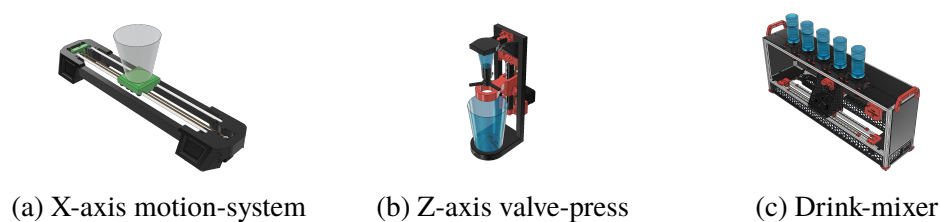


Figure 2.5: CAD renders of drink-mixer and motion assemblies.

2.4 PCB development

Three printed circuit boards (PCB) for stepper motor control have been developed by me. Two of the PCBs are shields based on the Raspberry PI 2W computer, and one is based on an STM32 controller. The first PCB-shield developed, *the Alchemist*, was used in the drink-mixer project for control of stepper motors and end-stops. The second PCB shield developed, *Tinky*, is an STM32-based controller board for two stepper motors. The purpose behind the development of Tinky was to gain experience in STM32 circuit integration and STM32-Cube-IDE software. Thirdly, *Tinkerer 1.0* was developed for larger-robotic projects supporting up to five stepper motors. Tinkerer is used in the robotic arm and is what makes the robotic arm project possible. Tinkerer is a evolution of the Alchemist PCB used in the drink-mixer project, where as Tinky is an STM32 take on the Alchemist PCB. All experience obtained through development and integration of these PCBs are critical to the robotic arm project. Figure 2.6 below shows KiCad renders of the developed PCBs.

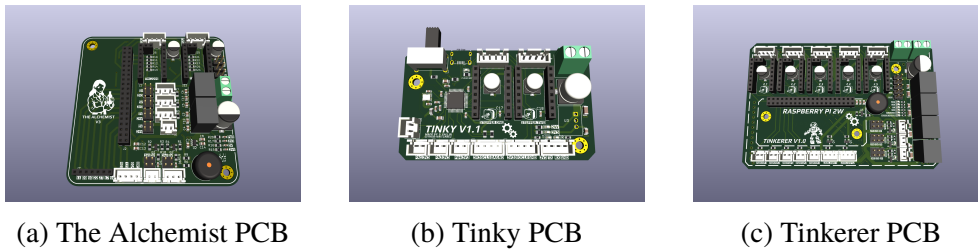


Figure 2.6: 3D-view of developed PCBs.

2.5 Summary

Involvement with multiple scratch builds and contributions in Voron Design related projects gives a strong foundation in Mechatronics. Additionally, acquired knowledge in the construction of mechanical systems, computer-aided design (CAD), and 3D-printing is directly applicable to a robotic-arm project. Successful self-developed projects like the drink-mixer and various PCBs designed and integrated to projects validate the background experience further.

Chapter 3

Methodology

This chapter covers the methodology used for the design, construction, and testing of the robotic arm. All methods used stem either from earlier experience, intuition, or through research.

3.1 Design inspiration

Arm joints of robotic arms are either supported on one side or both sides. While arm joints with single side support are aesthetic, they're challenging to implement properly. The prototype robotic arm design was based on single side support robotic arms like ABB GOFA CRB 15000 [9]. With issues discovered in section 2.1.2, it was realized that dual support for arm joints would work better with budget hardware.

A compelling design was found for the next revision of the robotic arm when analyzing ABB IRB 120 [10]. The IRB 120 design utilize dual support for all arm joints and have an offset for the second arm joint which allow it to fold. The folding property was deemed important for a design using micro-switches as end-stops and gives the arm a natural, relaxed homed position with no strain on motors. Without a relaxed homed position, idle holding torque of motors must be high to prevent movement. Although gear reductions amplify holding torque of motors, it also slows down movement significantly. As result, the final design was based on IRB 120 and its arm joint offset.

3.2 Designing for open-source

The collaborative power of open source comes from the community. In order to attract contributors, much time was allocated to design. Given that similar DIY projects exist online, design was identified as a key field to attract new contributors. Five key design goals were chosen and implemented as a result.

The first goal is that completeness catches attention. In do-it-yourself (DIY) projects, completeness is often overlooked as focus is put on function over form. As a result, DIY projects often end up looking unfinished. Small additions like designing a proper enclosure for the electronics can make a DIY project look professional, and by extension, more desirable for others to craft.

The second goal is to design a product that looks coherent. To do this, repetition of patterns was used. This approach, if applied with restraint, can create a professional look that appeals to a broader audience.

Third goal is to supply a defined color scheme which offers a balance between customization and visual consistency. By distinguishing dominant and accent-colored 3D-printed parts, builders can personalize their units with minimal effort while maintaining a cohesive look. This technique is especially effective in 3D-printed projects where many visible parts are present and a wide range of filament colors are from.

Fourth goal is support for modifications and encourage experimentation, unused anchor points in the form of heat-set-insets were included throughout the design. These predefined attachment locations enable users to add custom modifications without altering the core structure, enhancing the project's flexibility and long-term usefulness.

Fifth goal is to build an all-in-one robotic-arm solution. DIY projects that involve robotics arms are often split into two assemblies. One robot arm assembly, one electronics assembly, and wiring between the two. At a glance, such a setup may look awkward to handle. A solution to this is to integrate everything into a single unit so that electronics reside in the base of the arm instead. The intention here is to reduce the visual complexity and increase portability by merging two assemblies into one.

Four out of five design goals were the result of analyzing the design of 3D-printers developed by the open-source initiative *Voron Design* [1]. Designs by Voron Design always come in a two-tone color-scheme, room for modification, repetition of patterns (hexagons), and a complete design. The fifth design goal was identified while looking at other DIY robotics projects for points of

improvement.

3.2.1 Print-ability

Printability is the concept of designing 3D-models such that they can easily be 3D-printed. Print-ability can not be overlooked in open source designs as builders have different 3D-printers and conditions. Printability can be hard to implement in complicated designs. Such designs may need to be divided into multiple 3D-models, complicating the designs further. Another important design rule has to do with part orientation. It turns out that the orientation of the 3D-model being printed affects its properties. The impact of orientation is explained in section 3.3. All designs in the project were carefully designed with printability in mind.

3.3 3D-Printing

Regardless of the 3D-printing technology used, the workflow stays the same. For this project, the technology used is fused deposition modeling (FDM), which is a popular choice among hobbyists. Moving from 3D-model to a printable part is done through a *licer* [11]. First, the 3D-model is imported into the slicer software. Inside the slicer, the model is split into thin slices that, once stacked, result in the 3D-model.

Apart from slicing, the slicer software is also responsible for printer parameters such as layer width, temperatures, and speeds. Once done, the slicer returns a G-code file that contains all coordinates and speeds that the printer must follow to replicate the 3D-model. Finally, the G-code file is sent to the printer. The printer can now follow the movements required to replicate the sliced 3D-model.

3.3.1 Choosing material

The benefit of FDM technology, apart from its popularity, is the variety of plastic materials available. Plastics such as PLA, PETG, and ABS are the most common and can be bought for a reasonable price. To keep the cost and bar to entry low, it makes sense to pick either of the mentioned plastics above in an open-source project.

Since the motor mounts for the stepper motors are 3D-printed, a non-negotiable requirement is heat resistance. Stepper motors can get very warm once enabled by the stepper drivers. Operation of the arm requires stepper

drivers to be enabled at all times. When stepper drivers are enabled, current flows through the stepper motors, which increases their holding torque but also produces heat. Because of heat resistance, both PLA and PETG can be discarded as alternatives. ABS, on the other hand, typically has heat-resistance above 100° before showing signs of deformation. As a result, ABS is the plastic of choice for the project.

3.3.2 Engineering-grade materials

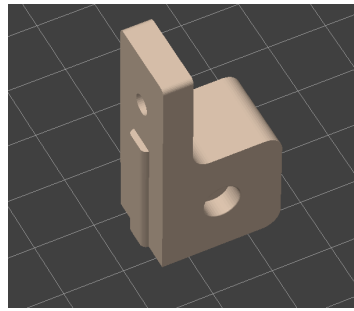
Engineering-grade materials, unlike budget alternatives mentioned in section 3.3.1, are also strong contenders for a robotics-arm project. The two most common engineering-grade materials are nylon-based blends or polycarbonate blends. Engineering-grade materials typically require very high nozzle, bed, and chamber temperatures to print. A hardened nozzle may be required as some blends have abrasive additives such as chopped carbon fiber, which, over time, destroy common brass nozzles.

Engineering-grade materials are discarded as alternatives for this project for the following two reasons:

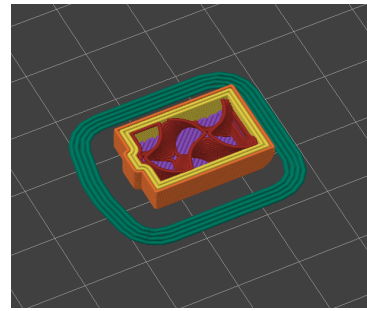
1. High cost, sometimes three to four times as expensive as budget alternatives.
2. High requirements for the 3D-printer of the builder, which can block the influx of new builders for the project.

3.3.3 Slicer parameters

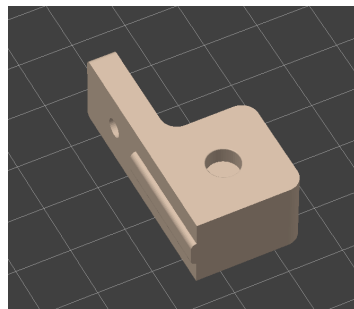
Slicer parameters that determine the output of a printed part are critical to get right. The infill parameter quickly reaches diminishing returns and can add unnecessary weight [12]. The infill density chosen for the parts of the project varies in the range of 15% - 25%. Wall thickness and layer width parameters are what give durability and strength to a part. Part orientation in the slicer software also affects the strength of a part [13]. Wall thickness and part orientation can be solved in the slicer alone, while the layer-width parameter is tied to the nozzle diameter of the 3D-printer. A good part orientation is characterized as the orientation that gives the largest cross-sectional area for a slice. Examples of correct vs incorrect orientation can be seen in Figure 3.1 below.



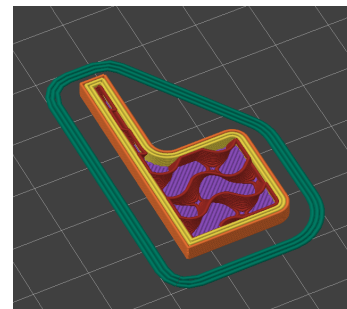
(a) Incorrect orientation



(b) Sliced view of incorrect orientation



(c) Correct orientation



(d) Sliced view of correct orientation

Figure 3.1: Slicer examples showcasing correct vs incorrect orientation.

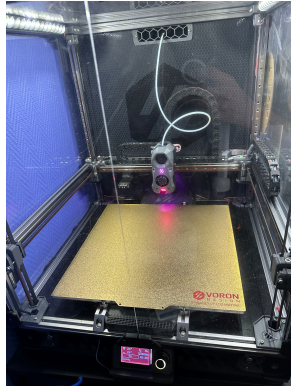
3.3.4 Required amount of plastic filament

The total filament used in the project is less than three kg. For a single color build, it's possible to print all parts with two kg. Although two kg suffice, it's not recommended as it leaves little room for print errors. The plastic used in the project is eSUN ABS, which is a budget brand. ABS+ compared is an altered version of ABS through additives. These additives provide better part durability, build-plate adhesion, and produce less odor when printing [14]. Both ABS and ABS+ produce harmful volatile-organic-compound fumes [15] and must be printed in a well-ventilated area or an enclosed area with filtration.

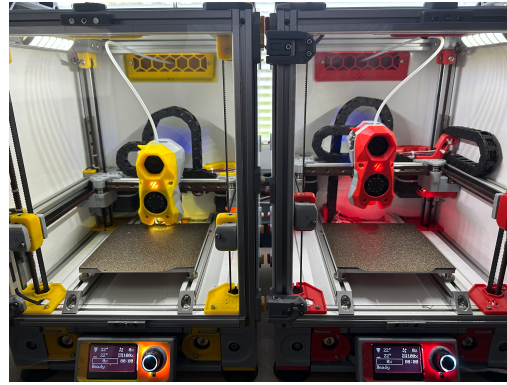
3.3.5 3D-printers used in the project

3D-printers used in the project are three Voron 2.4 printers. Voron printers are optimized for ABS plastics as they have an enclosed design. An enclosed design traps heat in the print chamber and reduces the temperature gradient between the heated build plate and printed part. Furthermore, the printers are fitted with convection fans to improve heat distribution inside the print

chamber. The convection-fan module is fitted with an activated-charcoal pellet-tray through which the ABS-fumes are filtered through [16]. Voron 2.4 printers are named after their printable volume and can be built in different sizes. Printers used can be seen in Figure 3.2 below.



(a) Voron 2.4 350mm³



(b) Two Voron 2.4 150mm³

Figure 3.2: FDM 3D-printers used in the project.

3.4 Gear reductions

To meet the torque requirements of a robotics arm, gearboxes are necessary. A gearbox trades speed for torque. Motors that move joints carry weight, and it's important to gradually reduce gear ratios the further out a motor is located on the arm, to preserve operational speed and to give more torque-room for lifting objects. *Backlash* is a mechanical parameter commonly specified for gearboxes. Backlash is a consequence of gear-meshing inside a gearbox and is the angle at which the gear output can be moved freely without moving other gears [17]. In figure 3.3 below, a 2:1 gear-reduction is shown. The large gear is the output, and if visually examined, it's clear that teeth-meshing isn't perfect, resulting in free play known as backlash.

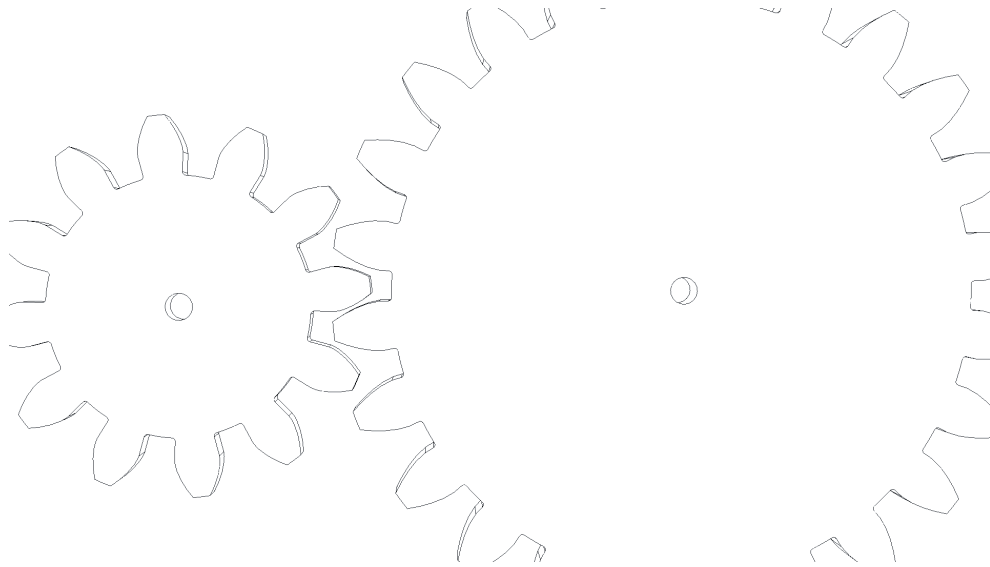


Figure 3.3: CAD render of a 2:1 gear-reduction.

3.4.1 Belted gearboxes

Belted gear reductions were a strong contender for the project. Belted gear reductions allow for the displacement of the driving motor, which is beneficial. Motors that are located closer to the base of the robotics arm reduce the torque required for movement. Belted-reductions have low backlash when properly tensioned.

Disadvantages of belted gear reductions stem from additional complexity and the belt itself. Firstly, belts stretch over time and require a tensioning mechanism thoughtfully placed so tension can be adjusted with ease. Untensioned, the belt starts to slip. Secondly, toothed belt loops can be hard to find in appropriate sizes. Lastly, large reductions become complicated to implement and to make compact. Despite the flaws, it's still a common solution in do-it-yourself (DIY) robotics, as toothed pulleys can be 3D-printed. In Figure 3.4 below, a simple 5:1 belted reduction is visualized. Note that the axis that hosts the 80-tooth pulley can be displaced further away by using a longer belt.

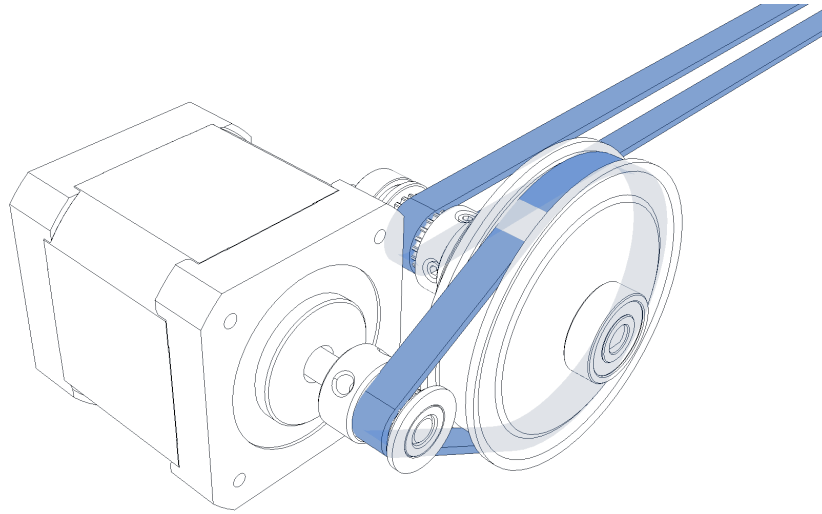


Figure 3.4: CAD render showcasing a belted reduction driven by a stepper motor. The 16-tooth pulley is driving an 80-tooth pulley through the help of a timing belt, resulting in a gear reduction of 5:1.

3.4.2 Cycloidal gearboxes

Cycloidal gearboxes are another common gearbox used for DIY robotics. Use of cycloidal gearboxes is appropriate when large reductions are required in a small form-factor, making them an excellent choice for joints that require high gear ratios. Like belt-reductions, cycloidal gearboxes provide low backlash.

The downside with cycloidal gearboxes is complexity. Many non-3D-printed parts are required, and the 3D-printed parts of the assembly are hard to design. Although 3D-printed, fully functional, and with great performance, cycloidal gearboxes exist, cycloidal gearboxes are not an option for time-constrained projects due to their complicated design.

3.4.3 Planetary gearboxes

Planetary gearboxes are ideal for many projects as the gearboxes are efficient, can be made compact, and are suitable for both low and high gear reductions. Stepper motors can be specified with integrated planetary gearboxes. A gearbox solution with the motor and gearbox as a combined unit is a compelling choice for time-constrained projects.

Unfortunately, planetary gearboxes with low backlash come at a high price

tag. Budget solutions of stepper motors with integrated planetary gearboxes come with more backlash. Backlash is devastating for precision, especially since the displacement error grows over the length of the arm. The backlash problems are examined in detail in chapter 5 of the thesis.

3.5 Building a mathematical model

In this section, a mathematical model for the robotic arm is built. Robotic arms have their own set of parameters and designs that need to be accounted for when constructing mathematical models. The takeaway here is that no mathematical model fits all robotic arms.

3.5.1 Measuring the robotic-arm

When building a mathematical model for a robotic arm, lengths of joint assemblies are required. Lengths for robotic arm joints can be found below in Table 3.1. Joint names are based on the naming used in section 4.2. Note that only arm joints have a length.

Table 3.1: Arm joint lengths.

Joint assembly	Length (mm)
First arm joint	200
Second arm joint	125
Third arm joint	75
Static-displacement of second arm joint	66

Measurement of arm joint lengths is not trivial. A consequence of the chosen design introduces a static displacement of the second arm joint, seen in table 3.1 above. Figure 3.5 below gives a clear view of all measurements and static displacement.

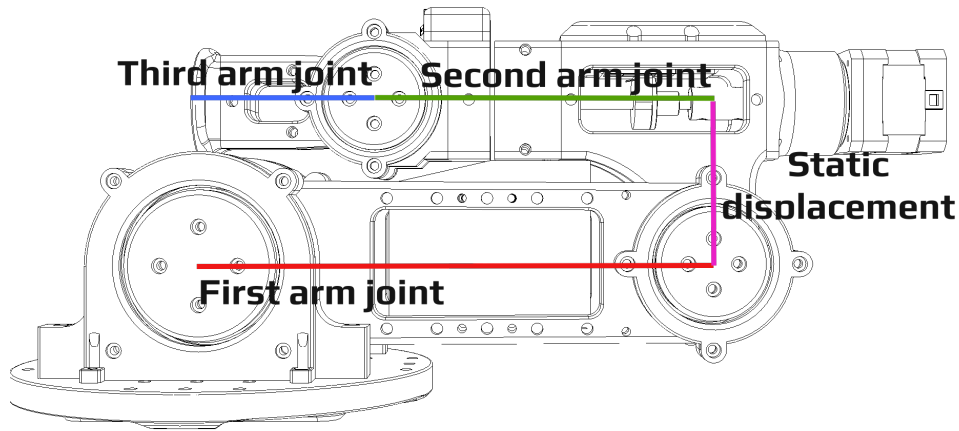


Figure 3.5: CAD render showcasing how lengths in table 3.1 was determined.

While lengths of arm joints and static displacement were presented in table 3.1, there is one length missing - the end-effector. End-effectors are devices mounted to the very end of the third arm joint to allow interaction with objects. An end-effector would, for instance, be a grip-claw.

Lengths of end-effectors must also be taken into consideration when constructing a mathematical model. A non-curved, straight end-effector that points in the same direction as the third-arm joint is easy to add to a mathematical model - it would simply be treated as if the third arm joint were longer. Since no end-effectors were developed in this project, their impact on the model is discarded.

3.5.2 Forward kinematics

Forward Kinematics (FK) is a mathematical model in which known angles are used to calculate the coordinates of the end-effector. Simply put, FK answers the question *"Given a set of joint angles, what is the position of the end-effector?"*. Analysis of FK were done using *Denavit and Hartenbergs* (DH) method [18] which involves five steps:

1. Number the joints and links of the robotic arm.
2. Attach a local coordinate reference frame for each joint following DH convention.

3. Calculate the DH parameters for each link.
4. Calculate the matrix of homogeneous transformation for each link.
5. Compute the kinematics equations of the robotic arm.

DH parameters were obtained through specifications for a similar robotic-arm construction ABB IRB-120 [10], and were modified to suit the project. The DH parameters were used to construct homogeneous transformation matrices sequentially for each link, which were then multiplied together. The result is the complete transformation from the base to the end-effector, given a set of joint angles. The homogeneous transformation is well documented in the literature and can be treated as a black-box. What's important are the four DH parameters the transformation takes to compute the final position of the end-effector. Table 3.2 below lists the found DH parameters for the robotic arm. Note the distance parameters a and d which stem from measurements in Table 3.1.

Table 3.2: DH-parameters for robotic-arm

DH Parameters	θ (degrees)	α (degrees)	a (mm)	d (mm)
Joint 1	0	-90	0	0
Joint 2	0	180	-200	0
Joint 3	-90	90	-66	0
Joint 4	0	90	0	-125
Joint 5	-180	-90	75	0

After computing FK with parameters listed above, the result of homing angles can be seen in Table 3.3 below. The complete code for FK is found in Appendix D.

Table 3.3: End-effector position using FK

Coordinate	(Distance (mm))
X	-75
Y	0
Z	141

Figure 3.6 below shows how the results in Table 3.2 relate to the model in a homed position. Note the reference coordinate system, which does not take into account the height of the base assembly.

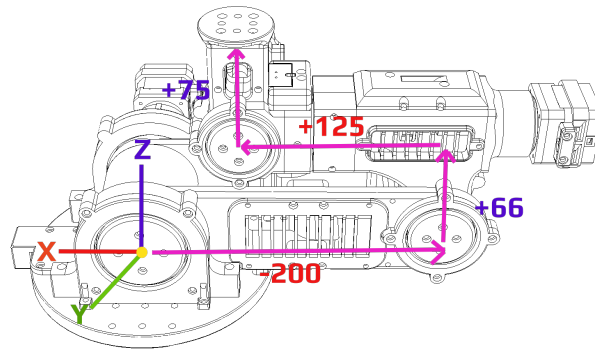


Figure 3.6: Location for end-effector in homed position.

3.5.3 Inverse kinematics

Inverse Kinematics (IK) is the opposite of Forward Kinematics. In IK, the object to interact with is the starting point, and angles are worked out from the end-effector towards the base of the robotic arm. IK answers the question *"Given the position of an end-effector, what are the joint angles?"*. IK was not implemented in this thesis, but is worth acknowledging as it's part of future work. With IK, the position of objects can be given to the arm, and the angles to grab the object are worked out. For multi joint robots, multiple solutions occur as several linear combinations yield the same end-effector position, which must be dealt with in code.

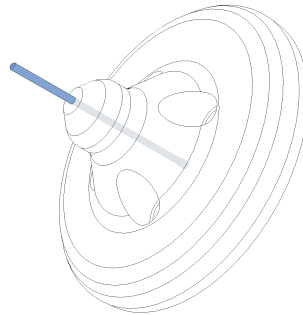
3.6 Testing

This section is the main scientific contribution of the thesis. It explains the methods and test setups used for evaluation of the robotic arm. Looking at commercially available similar robot offerings [19] [20] reveals that repeatability and load capacity are key parameters to identify. Additional parameters of interest are deflection, accuracy, reach, and backlash.

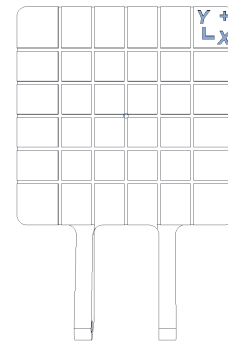
3.6.1 Repeatability test

Repeatability is the measurement of how close the robotic arm gets to a known location after movement. As result, repeatability isn't influenced by theoretical

positional values but rather actual positional values. As such, this test does not account for static errors but rather varying errors such as backlash. To measure repeatability, a self-made test was constructed to mimic the game *dart*. To represent the arrow, a needle was used. The needle was fastened in a 3D-printed end-effector. The target-plate is a 3D-printed design with 1mm wide slits and 1cm spacing between them. The center of the target plate is considered the origin of a 2D plane. Plane-coordinate-reference and sign-reference were embossed into the upper right-hand corner of the target plate. Test tools constructed can be seen in Figure 3.8 below. Figure 3.7a represents the dart and Figure 3.7b the target plate.



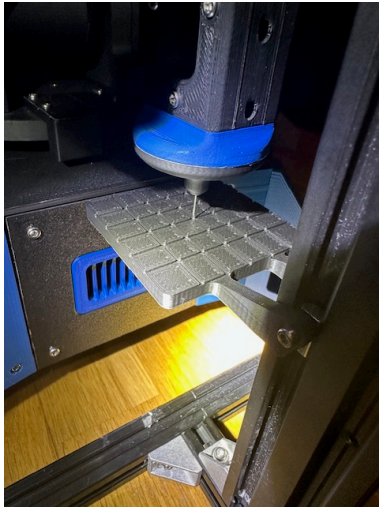
(a) CAD render of needle-end-effector. The needle is highlighted in blue.



(b) CAD render of target-plate. The coordinate and sign reference is highlighted in blue.

Figure 3.7: CAD renders of self-made tools.

A test setup rig was built out of aluminum extrusions. To ensure that the rig stays in place, the robotic arm was placed on top of the rig, locking the two systems together. In Figure 3.8 below, you can see the two setups used for horizontal-repeatability and vertical-repeatability. In both pictures, the needle is centered in the middle of the target.



(a) Horizontal repeatability-test setup



(b) Vertical repeatability-test setup

Figure 3.8: Setup for repeatability-tests with tools presented in fig 3.7. The needle is centered on the target plate in both figures.

After the needle centering on the target plate is done through manual movement of the robotic arm, the robotic arm is displaced from the target plate by altering the angles of all axes a fixed amount. Angles used for displacement in both repeatability tests were 10° , 20° , 30° and the homing-position.

Needle-displacement for each test was measured manually using a digital caliper, and test angles were probed five times each. Because of self-made tools, potential errors in setup, and manual readings, the repeatability test can only give a limited result. Slits embossed into the base-plate ensures 0.5mm repeatability-range at center of the base-plate and can without calipers visually be identified.

3.6.2 Load capacity test

Load capacity of the robotic arm is defined as the max weight that can be moved without any stepper motor losing steps. With an open-loop stepper motor configuration, lost steps yields static errors that persist until the robotic arm has been homed. For measurement of load capacity, weights are mounted on to the end-effector of the robotic arm and the robotic arm performs movement of the weight. The position of interest for such test is the fully extended position such that maximum torque is applied to all motors. By gradually increasing the weight until a motor skip steps, load capacity can be identified. Additionally, the weakest motor can be identified which is crucial

for further improvements. Load capacity tests for stepper motors are well documented and the test method used was improvised based on that.

3.6.3 Accuracy test

Accuracy is determined by how close the robotic arm gets to its theoretical position for a set of joint angles. Contrary to repeatability, accuracy-test reveals static errors such as compound backlash or step losses in different states. With computer-aided design (CAD) software, joints of the arm can be adjusted, and through doing so, reference values for an accuracy test can be obtained. Coordinates for the arm end-effector can also be obtained through a forward-kinematics mathematical model derived in section 3.5.2. Note that repeatability tests measure how close the robotic arm can get to the same spot (same commands) while accuracy measures the error to its theoretical position with given coordinates.

To determine the real position of the robotic arm without the use of professional tools is challenging. Using hand tools to determine its position introduces errors, which render such a test useless, instead, pictures were taken and imported into a CAD program and scaled to estimate deviations. This test is also self-made and takes inspiration from thieves copying keys using CAD, 3D printing and a photo of the key [21]. Figure 3.9 below showcases an example of position estimation. All measurements used can be found in appendix E, note that perspective impacts the result.

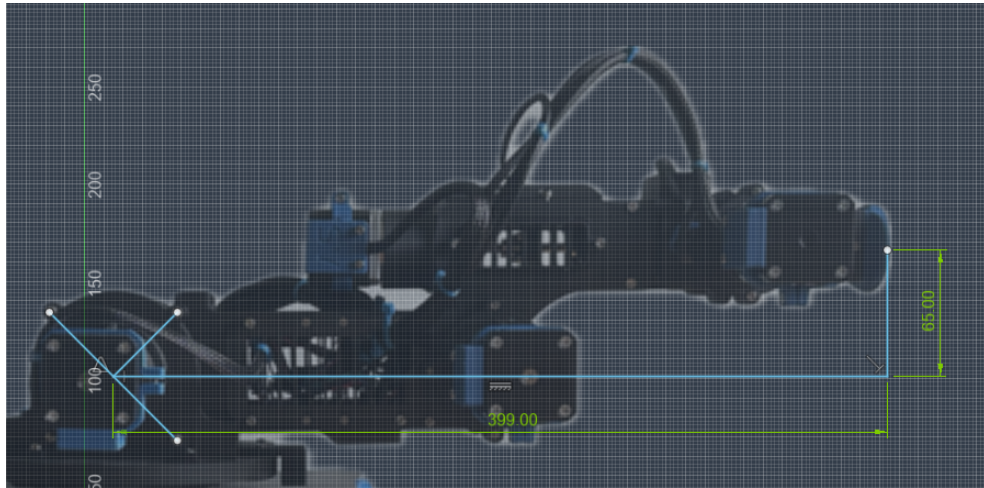


Figure 3.9: End-effector position estimation from scaled photo.

3.6.4 Deflection test

Deflection is a measurement of structural deformation under load. By extending the robotic arm to its full horizontal reach and placing a dial gauge under it, the arm can be loaded with weight, and deflection can be read on the dial gauge. Deflection tests using dial-gauges are well documented and simple to setup, and the test setup used was improvised. A 3D-printed box was constructed and mounted to the robotic arm to allow for weights to be placed. Weights used were AA batteries, stepper motors, and bearings. Each item was weighed on a kitchen scale prior to placement on the robotic arm. Figure 3.10 below showcases two different setups considered for the test. The first setup involves a tripod-mounted gauge using a 3D-printed mounting solution, and the second setup is simply using books and papers to get the distance correct. The first setup was discarded due to height inconsistencies related to the tripod itself.



(a) Discarded tri-pod setup



(b) Used setup

Figure 3.10: Setups for deflection test. Figure 3.10a showcases the discarded tri-pod setup, and Figure 3.10b showcases the used setup for testing.

3.6.5 Backlash test

Backlash is defined as the free play at the output of an gearbox, and is represented as angle. This angle can be seen as a *backlash range* which includes all unintended positional errors due to the gearbox play. To identify backlash in a robotic arm, the easiest way is to enable all stepper motors so

that they hold their positions. With motors on, each joint can be moved in the backlash range until the holding torque of the motor can be felt. Once the end or start of a backlash range has been identified, the joint is moved in the other direction until stopped by the holding torque of the motor. The angle between the start and stop of the backlash range is the backlash for that specific gearbox. By manually pushing an axis into its max backlash range and then commanding movement in the opposite direction, a good backlash estimation can be determined. This method exploits reversal backlash [22] and the tests were improvised utilizing stepper precision instead of a dial gauge.

3.6.6 Reach simulation

It's common that manufacturers showcase a picture of the robotic arm's reach in the XZ-plane. The XZ-plane gives an overview of which points can be accessed by the robotic arm when seen from the side. With the help of the derived mathematical model for forward-kinematics in section 3.5.2, points in space can be obtained for a set of angles. By sweeping through angles in small increments, points can be obtained and plotted. A birds-eye XY-plane plot is not particularly interesting for the robotic arm as it simply yields a round circle with a radius of 400mm. A XY-plane reach plot would only hold significance with respect to height (Z) restrictions and requires a predefined work environment.

Chapter 4

Design and Construction

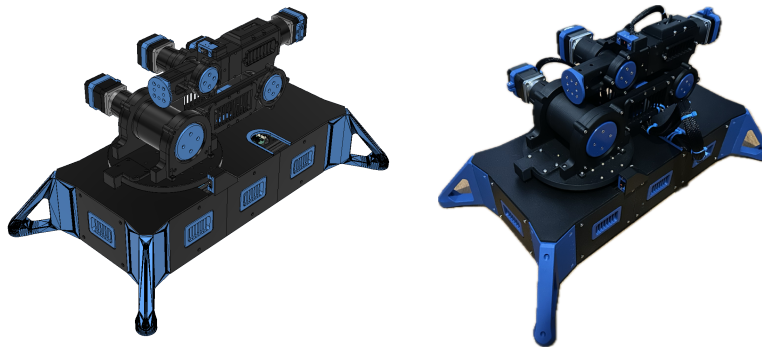
4.1 Design

With the realization that the prototype robotic arm had fundamental issues, the base of the assembly was kept, but the arm was redesigned. The new design was dimensioned down to 40cm total length, and arm joint length proportions come from the author's arm. To improve stability, the new arm utilizes a dual bearing support for all axes. New bearings, steel rods, and couplers were bought and dimensioned down to reduce weight. With smaller hardware came room for shrinking the size of 3D-printed parts. 3D-printed parts were designed simply and were given slight fillets to hide the simplicity. Arm joints had their cross-sectional area gradually shrunken to distribute weight in the full arm properly. The second arm joint was given an offset design to allow the arm to fold, which simplifies a necessary control feature called *homing*. Homing is elaborated on in section 4.3 and 4.4. Figure 4.1 below showcases the new design of the robotic arm.



Figure 4.1: CAD render of project robotic-arm

Through small but effective design choices, the new robotic arm was not only prettier but solved all issues that were discovered with the prototype. The complete final assembly of the project can be seen in Figure 4.2 below. The arm is in its folded position in both models.



(a) Final CAD assembly

(b) Final real assembly

Figure 4.2: Final design of the robotic arm. The arm is resting in its folded position.

4.1.1 Issues encountered

The robotic-arm build was not problem-free, and a few issues were encountered. The most pressing issue was related to the material of choice. ABS plastic is notorious for its small but sometimes relevant amount of shrinkage. Joints of the build that key into bearings resulted in being too

small, resulting in noticeable play in the joints. The solution to ABS shrinkage was to over-dimension affected features slightly. Figure 4.3 below showcases ABS-shrinkage in an earlier revision of the first arm joint. Note that shrinkage affects both features that key into bearings and house bearings.

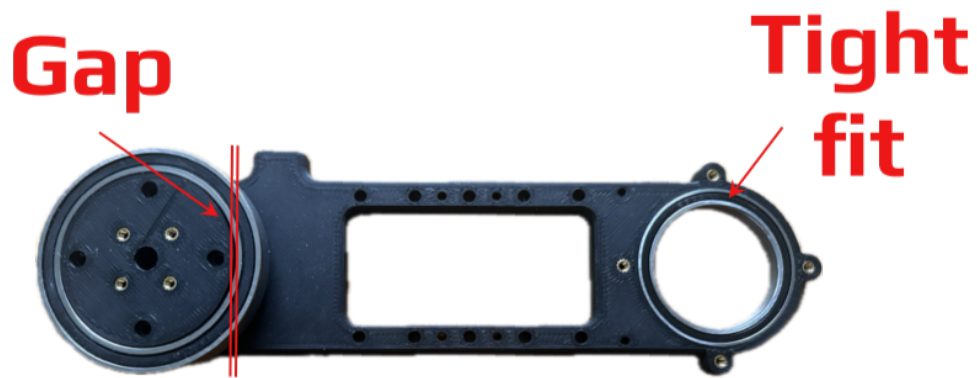


Figure 4.3: Tight and loose fit of a non-shrink compensated ABS-part.

Another issue was the design for the forwarding of rotational torque of the fourth stepper motor for rotation of the third arm joint whilst going through the second arm joint. The assembly had to be compact, rigid, and have a way to click the end-stop used. Figure 4.4 below showcases the final design of what's regarded as the most complex assembly of the robotic arm. The exploded view of this assembly can be seen in Figure 4.9.



Figure 4.4: Torque transfer assembly with dual-bearing support and a click-wheel for end-stop.

4.2 Mechanics

This section puts emphasis on hardware and assemblies that contribute to the stable and precise movement of joints. Stepper motors used in the project have budget-integrated gearboxes with planetary-gear reductions. The motors come with a backlash of $< 3^\circ$ [23], which causes positional errors. Motors and gear ratios were determined before the time frame of this thesis in the first robotic-arm prototype. As such, joint assemblies had motors re-allocated out of expected torque requirements.

4.2.1 Joint assemblies

Joints of the arm assembly were developed in order from the base to the end. This section examines each joint of the assembly, starting with the base joint. In Figure 4.5 below, the arm is color-coded to show which motor turns which joint and which assemblies are connected joints.

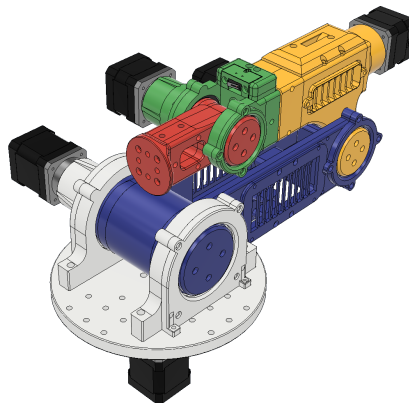


Figure 4.5: CAD render showcasing all five joints and their assembly color-coded.

The first joint developed rotates the base plate on which the arm is mounted, corresponding to the white assembly in Figure 4.5. The construction utilizes a *Lazy Susan* to handle smooth rotation and weight of the arm. Lazy Susans have many applications; the one used in the project is meant for a rotating stool.

The *Lazy Susan* is directly bolted onto the base aluminum-extrusion frame and the base plate. A flange-coupler is mounted onto the base-plate, and a

geared-stepper motor with 14:1 gear-reduction is pushed into a 3D-printed motor-mount. The D-cut shaft of the stepper motor is locked into place in the flange-coupler using two M4-Screws with thread-locker. Finally, the motor-mount is screwed into place on the same extrusions of which the *Lazy Susan* is mounted onto. An exploded view of the assembly can be seen below in Figure 4.6.

The base-plate rotation is constrained to a 180° rotation as it needs to physically click an end-stop.

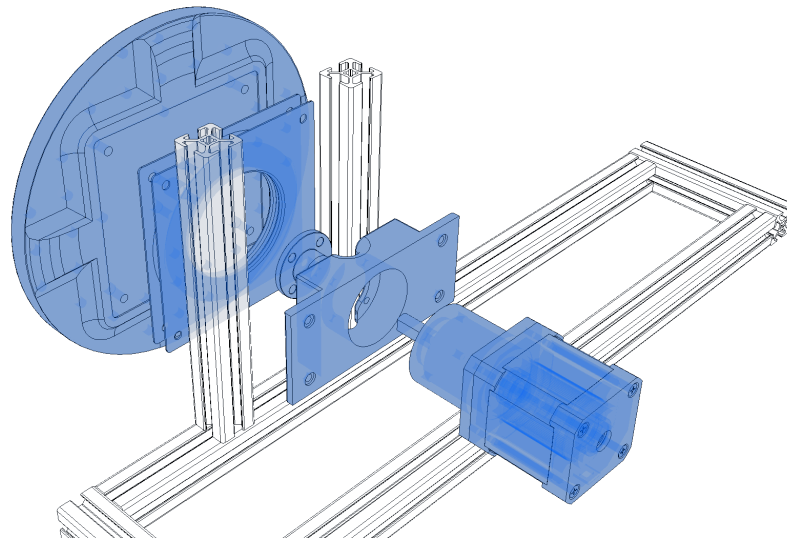


Figure 4.6: CAD render showcasing an exploded view of the first rotation joint mechanics. Highlighted parts are key for base-plate rotation (some aluminum extrusions are hidden to aid visibility).

The first arm joint is what carries the rest of the arm during operation, this is the blue assembly in Figure 4.5. The construction utilizes two 6812-2RS bearings to support the arm and provide smooth rotation. The outer side of the bearings is fastened into a 3D-printed housing, and 3D-printed arms lock into the inner side of the bearings. A large cylindrical 3D-printed spacer is put between the two arms to ensure the arms can't bend inwards. Each bearing-housing is mounted onto the base-plate using four M3 screws. The side where the motor is mounted gets a flange-coupler installed.

This arm requires the most torque as it moves the rest of the arm, a stepper motor with an integrated gearbox with a gear ratio of 51:1 is installed into a motor mount and a flange-coupler. Seen in Figure 4.7 below, the motor

mount also acts as part of the bearing-housing assembly. The D-cut shaft of the stepper motor is secured in place using two M4 bolts with thread-lock. To access screws for tightening the shaft onto the flange-coupler, the motor-mount hosts two holes allowing allen-key access.

The arm is constrained to 180° rotation as it cannot move through the base-plate on which it's mounted.

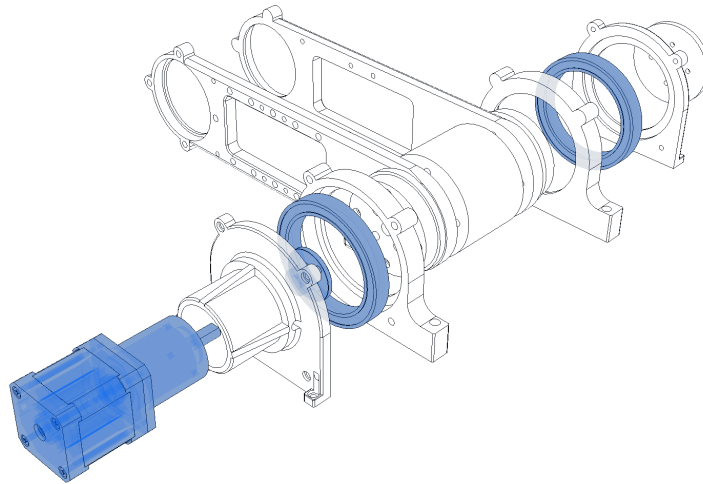


Figure 4.7: CAD render showcasing an exploded view of the first arm joint mechanics. Highlighted parts are key for rotation.

The second arm joint is similar in its construction to that of the first arm joint. This joint is the yellow color-coded assembly in Figure 4.5. The difference between the two assemblies is that the second arm joint uses a stepper motor with an integrated gearbox at 27:1 gear-reduction. The bearings are also smaller in this assembly, two 6809-2RS bearings are used here. Smaller bearings save weight and are used since this arm joint won't carry the same load as the first arm joint.

Comparing the highlighted parts in the second-arm joint exploded view in Figure 4.8 with Figure 4.7 clearly shows the similarities between the two joint assemblies. This joint is constrained to 180° rotation as it otherwise would crash into the first arm joint.

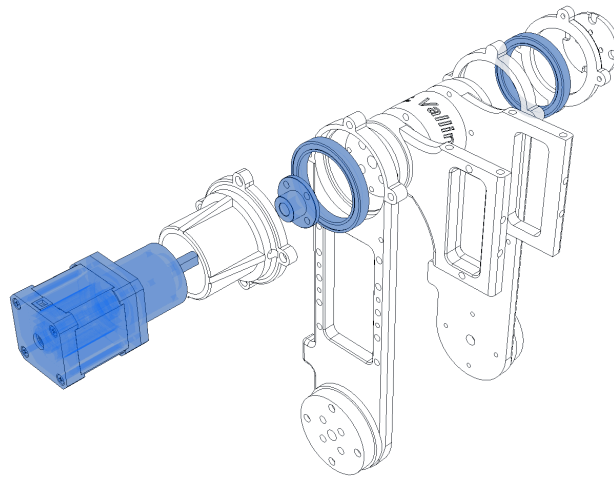


Figure 4.8: CAD render showcasing an exploded view of second arm joint mechanics. Highlighted parts are key for rotation.

The second rotational joint is the most complex design in this project and is a partial assembly of the second arm joint. This joint can be seen as the green-colored assembly in Figure 4.5. It uses an inline, spaced, dual 6805-2RS bearing support, sandwiched between three 3D-printed parts. Housing for the bearings is a two-part 3D-printed design. A stainless-steel rod with a 5mm diameter is used to transfer rotational torque produced by a stepper motor. The stepper motor used has an integrated gearbox with 27:1 reduction, but the motor itself is weaker than the motor used in the second arm joint. The stepper motor D-cut shaft is connected to a steel rod using a 6 mm-to-5 mm axis-coupler. The steel rod runs through the entire assembly and continues beyond the bearing housing, which stabilizes both the static and rotational parts of the arm. Two flange-couplers were used in this assembly to fasten the steel rod onto 3D-printed parts. Couplers are secured with M3 bolts and thread-locker.

The micro-switch used as an end-stop is activated by a 3D-printed part that rotates with the steel bar. Because of this solution, rotation is constrained to 180° for the axis. The micro-switch and parts of the second arm joint are hidden in Figure 4.9 below to highlight its complexity. The 3D-printed parts that activate the end-stop can be seen, however.

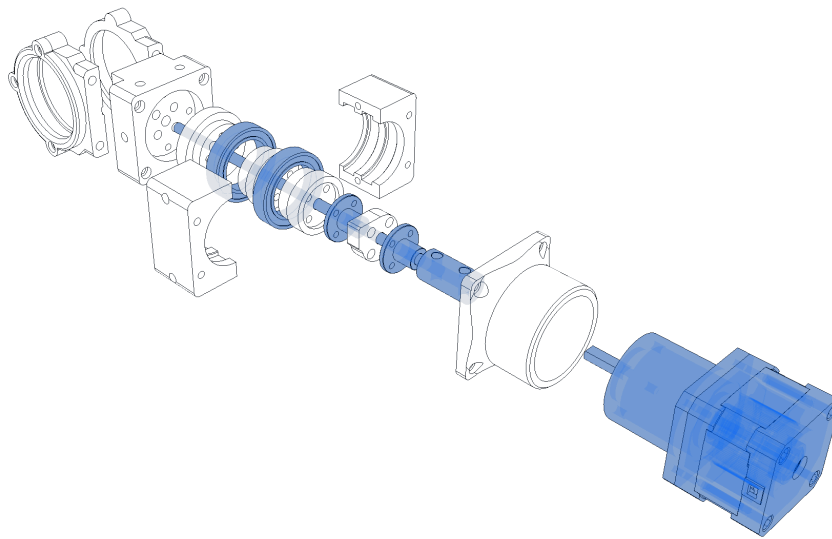


Figure 4.9: CAD render showcasing an exploded view of second rotation joint mechanics. Highlighted parts are key for rotation.

Similar to the first and second arm joints, the third arm joint utilizes a

similar construction. Just like the second arm joint, it uses smaller bearings and a smaller motor. The bearings used in the assembly are 6807-2RS, and the motor has an integrated gearbox with a 5:1 reduction. Like the other assemblies, this arm joint is constrained to 180°. The exploded view of this assembly can be seen in Figure 4.10 below.

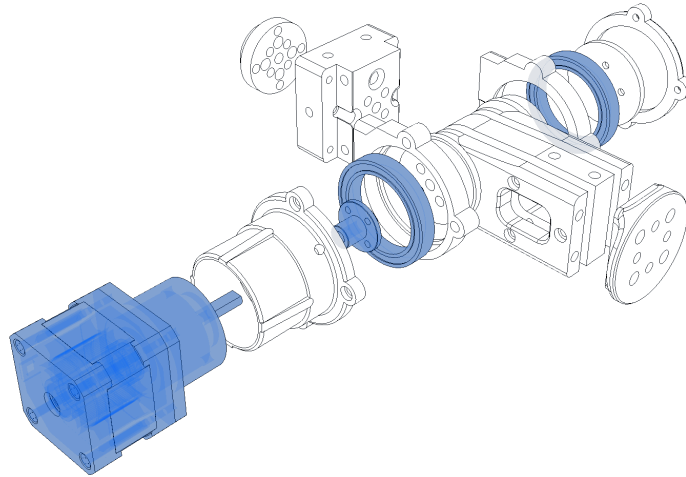


Figure 4.10: CAD render showcasing an exploded view of third arm joint mechanics. Highlighted parts are key for the rotation of the arm.

4.3 Electronics

In this section, the electronics used in the build are examined. Different components are compared, and the choice of electronics is discussed. The section starts with an overview of electronics used and continues diving deeper into more critical components of the project.

4.3.1 Overview

Electronics that power and control stepper motors sit inside the base of the robotic arm. Components are few, but each plays a key role. A 3D-printed rig holds the power supply in place, and slightly above it sits the Tinkerer 1.0 PCB shield in a 3D-printed support. An inlet supplies 230V AC to the power supply, which converts it to 24V DC. 24V DC is what powers Tinkerer 1.0. A tiny fan is installed to cool the electronics, but more specifically, stepper drivers

are installed on Tinkerer 1.0. Figure 4.11 below shows where electronics are located inside the base assembly.

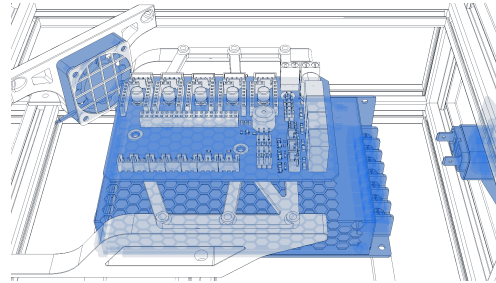


Figure 4.11: CAD render showcasing the electronics compartment of the robotic arm. Electronics is highlighted in blue.

4.3.2 Controller and PCB-shield

The controller for the robot arm is a Raspberry Pi Zero 2W. Raspberry Pi Zero 2W is a budget alternative to the more powerful Raspberry Pi 4 and Raspberry Pi 5. Although a cheaper alternative without wireless connectivity exists, it's deemed to be important for accessibility. Raspberry Pi Zero 2W has a 40-pin header of which 28 are general-purpose input/output (GPIO). Using software, the GPIO of the Raspberry Pi is used to interface with other components. The Raspberry Pi 40-pin header is mounted into a *Tinkerer 1.0* PCB.

Tinkerer is what's referred to as a *PCB-shield*. PCB shields are constructed for a specific controller to extend its capabilities. Tinkerer supports the Raspberry Pi family of controller boards with 40-pin headers. Tinkerer 1.0 was designed and constructed outside of this project, but has all the necessities required for a five-degrees-of-freedom (5-DOF) robotic arm.

4.3.2.1 Tinkerer 1.0 overview

The purpose of Tinkerer is to provide the necessary tools for most small robotic projects. The PCB itself is compact at 115 x 75mm, and attachments add to height only. Schematics for Tinkerer 1.0 can be found in the Appendix A. Figure 4.12 below shows the layout of Tinkerer. Note the 40-pin female Dupont header in the center, which interfaces with the Raspberry Pi 40-pin male Dupont header.

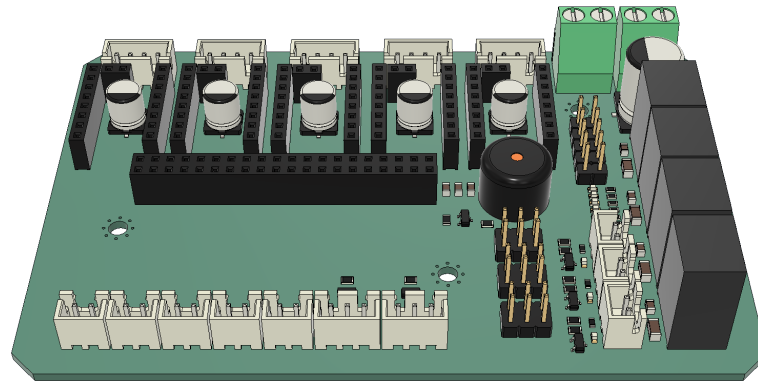


Figure 4.12: CAD render of Tinkerer 1.0.

Tinkerer maps Raspberry GPIO to new outputs and inputs. All GPIO pins of the Raspberry PI are directly tied to a specific output or input and this mapping must be taken into consideration when using Tinkerer. Stepper drivers for smaller motors typically come in a small PCB form factor with 16 to 18 leg-pins. Tinkerer hosts drop-in slots for PCB drivers, utilizing female Dupont connectors. Additionally, each drop-in slot hosts a 100uF 35V electrolyte decoupling capacitor for the motor supply voltage. Decoupling capacitors smooth noisy signals and voltage spikes. Furthermore, Tinkerer supplies five end-stop connectors - intended to be used in conjunction with a stepper driver. All five stepper drive drop-in slots and end-stop connectors were used in the robotic arm.

Tinkerer takes a 24V DC input voltage. To supply on-board peripheral electronics with adequate voltage, Tinkerer is fitted with four voltage rails. The voltage rails consist of 3.3V, 5V, 12V, and 24V. Each voltage rail has its own LED indicating the functionality of the responsible power supply. Voltage step-down is handled by onboard switching power supplies. Switching power supplies are troublesome to implement properly with components and layout - Tinkerer instead uses ready-made switching power supply ICs.

Tinkerer is equipped with two logic-level shifters. The purpose of logic-level shifters is to increase or decrease the voltage of a signal. Since the Raspberry Pi only sends and reads 3.3V logic-level signals, logic-level shifters

allow for 5V signals to be read. The logic-level shifters are not used in this project.

The three pulse-width-modulation (PWM) outputs of Tinkerer allow the Raspberry Pi to turn on and off devices that operate on a different voltage. PWM voltage is selectable through jumpers, and the voltage selection range is 5V, 12V, and 24V. PWM outputs are useful for controlling heaters and fan speeds, but can also be used as a relay. PWM outputs are not used in this project.

4.3.3 Stepper Motors

Stepper motors used in the project are *Nema-17* stepper motors. Nema-17 is a size classification based on the size of the motor's faceplate. A Nema-17 motor has a face-plate of 1.7 by 1.7 inches or 43.2 by 43.2 mm. Nema stepper motors are standardized hardware and commonly available as Nema-8, Nema-11, Nema-14, Nema-17, Nema-23, Nema-34 and Nema-42. Nema size classification is not directly tied to any of the characteristics of the motor, such as current rating, step angle, torque, shaft diameter, and motor length.

Although the Nema size classification holds no information about motor specifications, the classification can still hint at which application a stepper motor is intended for. For example, commercial hobbyist 3D-printers, CNC-machines, and laser-engravers typically use Nema-17 motors.

4.3.3.1 Stepper motors and feedback-loops

There's an important distinction to be made when integrating stepper motors into a project. Stepper motors can be either implemented in an open-loop or closed-loop configuration. Table 4.1 below showcases the difference between the two feedback-loop configurations.

Table 4.1: Feedback-loop comparison for stepper motors

Feature	Open-loop	Closed-loop
Feedback	No	Yes
Accuracy	Can lose steps	Step-loss detection and correction
Cost	Low	High
Setup	Simple	Complicated

Stepper motors are either sold as open-loop or closed-loop. Hardware exists to upgrade open-loop stepper motors to closed-loop. To keep the budget

low, Nema-17 stepper motors, which are used in the project, are open-loop and susceptible to step-loss. Step-loss for open-loop motors occurs when the torque limit of a stepper motor is exceeded. When motors have gear reductions mentioned in section 4.2, the torque required to cause a step-loss is significantly higher.

4.3.3.2 Stepper motor parameters

Stepper motors have important parameters to keep track of. Parameters like step-angle, number of coil windings, and rated current are critical. Step-angle refers to the angle at which the output shaft rotates on a step-pulse signal. For Nema-17 motors used in the project, the step-angle is 1.8° , which implies that 200 step-pulses are required for a full revolution.

Coil windings are tied to the number of wires coming out of a stepper motor. Coil-wire-pair is important to fully understand when installing a stepper motor. Stepper motors used in the project are all *bipolar*, which means two coil windings are used. Two coil windings result in two coil-wire pairs. A stepper driver has a dedicated pin-out for coil-wire pairs, and the driver datasheet must be examined before an installation. A quick way of identifying coil-wire-pairs is to short-circuit two wires and check if the motor shaft gets through to rotate. If the shaft gets tougher to spin, the two wires make up a pair.

4.3.3.3 Stepper drivers

This section explores two stepper drivers used in the project. Stepper drivers are electronic devices that directly interface with stepper motors to provide control. For Nema-17 stepper motors, drivers come in a small PCB format intended to be mounted onto a controller board or PCB shield like Tinkerer.

4.3.3.4 Amperage tuning and micro-stepping

Stepper drivers typically come with either dip switches or a potentiometer to toggle output current. Dip-switch tuning follows a predefined binary chart, and potentiometer toggle requires measurement of a reference voltage commonly labeled as V_{ref} . Different stepper driver models come with their formulas for calculating output current using V_{ref} .

Micro-stepping is a feature commonly found on stepper drivers. Micro-stepping technology increases the resolution of stepper motors, allowing them to step in micro-steps instead of full-steps. The result of micro-stepping is

that stepper motors gain precision and step smoothly. A trade-off for the feature is that it's more taxing for controllers, which have to send more step signals to stepper drivers. Micro-stepping is commonly found in four or more configurations and can be adjusted by pulling dedicated pins to HIGH or LOW on the stepper driver.

Current supplied to stepper motors is proportionally correlated to output torque. To maximize the performance of stepper motors, amperage-tuning can be used to match the rated current of the stepper motors. Running stepper motors beyond their rated current causes the motor to build up excessive heat, which damages it over time. Stepper drivers used in the project had their current adjusted to match the rated current of the stepper motor controlled.

4.3.3.5 Stepper driver TMC2226

Stepper drivers used in the project are TMC2226. These stepper drivers were in stock for the project. The Tinkerer PCB shield was designed around the TMC2226 pinout. Tinkerer was developed outside the time frame of the project, and as a consequence, it limits the range of compatible stepper drivers. TMC2226 drivers have enough amperage and are capable of handling the chosen Nema-17 stepper motors at their rated current. TMC2226 only has micro-stepping, and the lowest a TMC2226 can be configured is to eight micro-steps. This means that a geared stepper motor with 10:1 reduction and 1.8° step-angle requires 16,000 step-pulses to complete a full revolution. With many step-pulses required, a controller may struggle moving the motors quickly.

4.3.3.6 Stepper driver DRV8825

An attempt was made to speed up joints of the robotic arm by using DRV8825 stepper drivers instead of TMC2226. DRV8825 is a budget stepper driver that supports full-stepping, which reduces the number of step pulses required to rotate the shaft of stepper motors. The tradeoff is precision, but since the motors used have integrated gearboxes, resolution is already increased by the gear ratio of the gearbox.

Although the DRV8825 pin-out is similar to TMC2226 and is close to being a drop-in replacement, several issues were discovered after installation. Firstly, DRV8825 drivers produce excessive noise during operation, which doesn't give a positive impression of the overall product package. Secondly, the motors had uneven motion as if the drivers could not handle the speed

at which the step-pulses were sent. Further investigation into the usage of DRV8825 and troubleshooting was discarded due to time constraints.

4.3.4 Choosing power supply

The choice of power supply and wattage was determined through experience. Voron 3D-printers presented in section 3.3.5 use a 200W power supply and power seven Nema-17 stepper motors, fans, and a 50W heater-cartridge. Since the printer used has 345 documented print hours without electrical issues, it's safe to choose a 150W power supply for the robotic arm, which powers five Nema-17 stepper motors. The power supply chosen was Meanwell LRS-150-24, which is the same brand as Voron printer uses.

4.3.5 End-stops

Open-loop stepper motors have no feedback, and the controller has no way of knowing the position of an arm joint. End-stops are utilized to provide a known location for controllers. By letting a stepper motor move in the direction where the end-stop is located, once reached, the location of the joint is now known. This sequence is known as *homing*. A homing-sequence routine must, as a consequence, be performed on every startup of the robotic arm.

Micro-switches are a cheap and reliable solution for end-stops. Micro-switches are used in applications such as computer mice. Micro-switches are also commonly found in 3D-printers, CNC machines, and laser-engravers. Micro-switches can be found for $< 1kr$ and are a compelling end-stop solution for projects on a budget. Locations of micro-switches used in this project can be seen in ?? below.

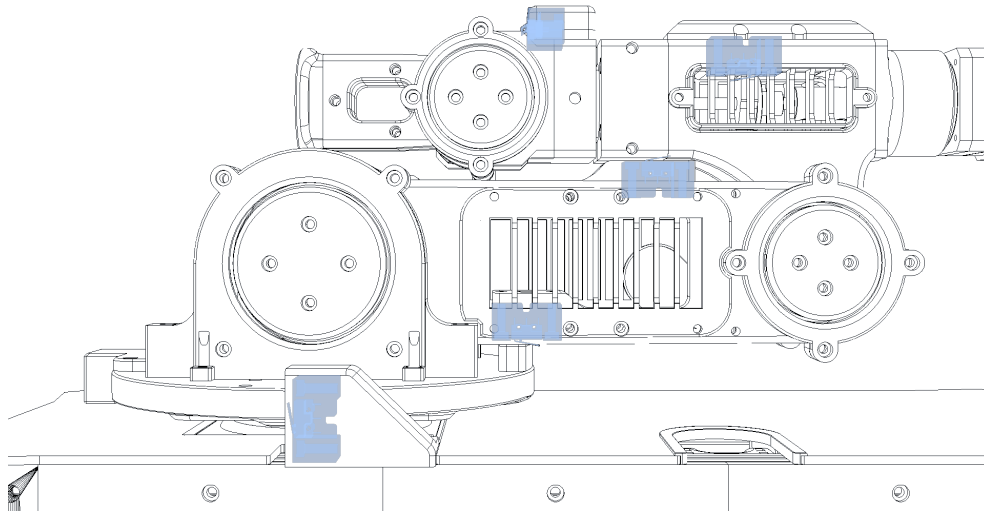


Figure 4.13: CAD render showcasing the location of the five end-stop micro-switches used for homing.

A micro-switch commonly hosts two inputs labeled *NO* and *NC*. In a *NO* configuration, the switch circuit is *normally open*, meaning that no current flows through the switch unless clicked. The opposite is true for *NC* configurations (*normally closed*)[24].

In case of broken wiring, an *NC* configuration would be read by the controller as if the switch clicked. This is critical as the homing sequence would stop immediately. For a *NO* configuration with broken wiring, the motors would home indefinitely, resulting in a crash.

4.3.6 Custom wire-loom

Construction of a custom wire-loom for stepper motors and end-stops was necessary to ensure full range-of-motion for the robotic arm. Wires used in the wire-loom are 0.25mm^2 silicone-insulated wires. The cross-sectional area was chosen to match the wires coming from the stepper motors. Silicone wires are cheap and flexible, suitable for moving applications like robotics. To provide protection and stability, the wire-loom was sleeved. Crimping is the process of adding end-fittings to wires. End-fittings are commonly pressed at two locations to provide a good fit. The crimping process starts with stripping insulation off a wire. Next, a press tool is used to press the first contact point so that it squeezes the insulated part of the wire. Finally, the

final contact point of the end-fitting is squeezed to secure the bare wire. At this point, the end-fitting is fitted, and wires are ready to be installed into a connector. Figure 4.14 below visualizes a correctly placed end-fitting before it is pressed into place. Note the two vertical bars on either side of the press-fitting, which are the contact points that grip the insulated and stripped part of the wire.

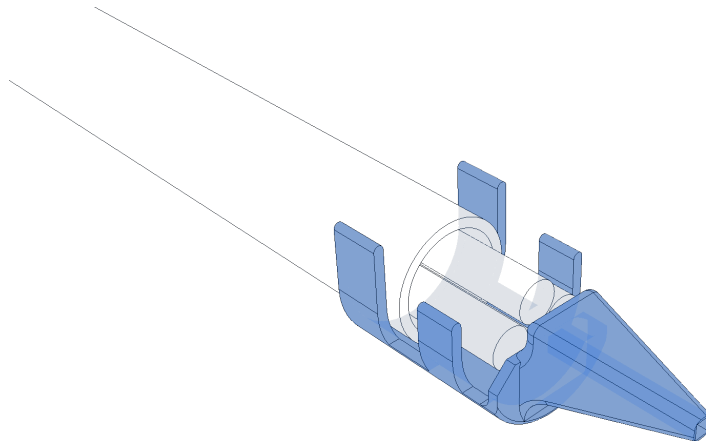


Figure 4.14: CAD render showcasing a correctly placed end-fitting on a wire - ready to be crimped. End-fitting is highlighted in blue.

Connectors are what end-fittings lock into, and they provide a repeatable solution for connecting and disconnecting electronics. For this project, Molex-Microfit 3.0 and JST-XH 2.54 connectors were used. Molex connectors were used to extend stepper motor wires, and the JST connectors to connect end-stops and stepper motors to Tinkerer. Molex connectors are used for connections that carry moderate current and require a strong connection. JST connectors are compact and can also carry moderate current, but are more suitable in a non-moving compartment due to their weak connection. Figure 4.15 below showcases the cable-loom routing from a top-down view.

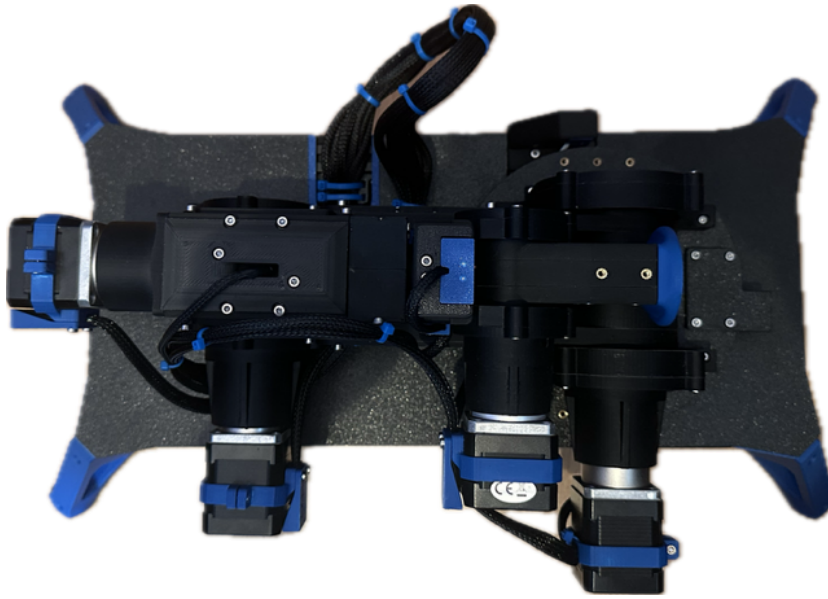


Figure 4.15: Cable-loom routing.

4.4 Software

In this section, you can read about how the programming language was chosen, how the controller was programmed, libraries used, GPIO mapping, and the program itself. Functions deemed important are explored in greater detail.

4.4.1 Programming language

The choice of programming language for software development was Python. Python offers good libraries for Raspberry PI GPIO and is quick to develop with due to its simple syntax and lack of redundancy. Low-level languages like C perform better than Python and but take longer to develop software with [25]. Because of time constraints, C development was not an option for this project.

4.4.2 Interfacing with the Raspberry

When flashing software for the Raspberry PI, it was set up to connect to local Wi-Fi. To save time and space, the legacy 64-bit Debian Bullseye operating system was chosen with a non-desktop environment. Interfacing

with the Raspberry Pi was done over SSH from a PC. VS Code programming environment was used with an SSH extension to simplify the coding process.

4.4.3 PCB Shield pinout

Explained in section 4.3, Tinkerer maps general-purpose input-output (GPIO) pins of a Raspberry Pi to new pins. To correctly interface with Tinkerer, a pinout file was created. Utilizing a Python dictionary key-value structure, pin numbers were mapped to letters to better aid readability. The pinout file is then imported by the main program to gain access to this pin-name mapping structure. The code snippet in listing 4.1 below shows pin-name mapping for step and direction pins for a stepper driver named S_A.

Listing 4.1 Example of pin-name mapping.

```

1 S_A = {
2     "DIR":20,
3     "STEP":16
4 }
```

4.4.4 Main program

In this section, the main program software is examined. Important functions and concepts are explained. Main program implementation as a whole is not covered here but can be found in the Appendix D.

4.4.5 Python library - pigpio

pigpio is a Python library that allows control of GPIO for any version of Raspberry PI. The library is C-based and is faster than other Python alternatives like RPi.GPIO [26]. The pigpio library is not a default installation on a Raspberry PI and must be installed by the user. Listing 4.2 below shows how the installation was done.

Listing 4.2 pigpio installation.

```

1 sudo apt update
2 sudo apt upgrade
3 sudo apt install pigpio python3-pigpio
```

To use pigpio, a background program - a *daemon* [27], must be started. A shell script was written to initialize the daemon prior to executing the main program. This way, the daemon is always active when the start script is run. The command to start the pigpio daemon is found listing 4.3 below.

Listing 4.3 pigpio daemon startup.

```
1 sudo pigpiod
```

GPIO pins of a Raspberry Pi can be set up either as inputs or outputs. For pigpio to function, inputs and outputs must be defined before reading or sending data. Initialization examples of GPIO using pigpio and the pinout-file presented in section 4.4.3 are shown in listing 4.4 below.

Listing 4.4 GPIO initialization using pigpio

```
1 import pigpio
2 import pinout as PINOUT
3 pi = pigpio.pi()
4
5 #Stepper A init:
6 pi.set_mode(PINOUT.S_A["DIR"], pigpio.OUTPUT)
7 pi.set_mode(PINOUT.S_A["STEP"], pigpio.OUTPUT)
8 #Endstop A init:
9 pi.set_mode(PINOUT.E_A["END"], pigpio.INPUT)
```

4.4.5.1 Movement functions

When implementing movement, there is a choice to be made - either movement is *absolute* or *relative*. Table 4.2 & 4.3 below showcases the behavior of both configurations.

Table 4.2: Absolute-positioning examples

Start-position (degrees)	End-position (degrees)	Input (degrees)
0	90	90
45	90	90
135	90	90
90	-45	-45

Table 4.3: Relative-positioning examples

Start-position (degrees)	End-position (degrees)	Input (degrees)
0	90	90
45	135	90
135	225	90
90	45	-45

Either implementation is valid and can often be toggled by a user in software. For the project, only absolute positioning was used. Absolute positioning makes writing predefined movement patterns easier.

To move a stepper motor, step signals must be sent to its stepper driver. The following code snippet in listing 4.5 below demonstrates how sending step-signals was implemented.

Listing 4.5 Sending step-signals using pigpio

```

1 def runA(steps, speed) :
2     for i in range(steps) :
3         pi.write(PINOUT.S_A["STEP"], 1)
4         delay.sleep(speed)
5         pi.write(PINOUT.S_A["STEP"], 0)
6         delay.sleep(speed)

```

The code snippet in listing 4.6 below shows how signal reading from end-stops was implemented.

Listing 4.6 End-stop GPIO reading using pigpio

```
1 def readEndstopA() :  
2     return pi.read(PINOUT.E_A["END"])
```

Mentioned in section 4.3.5, open-loop stepper motors require a homing sequence. Listing 4.7 showcases how the homing sequence for an arm joint was implemented. Note that the end-stop is wired in a normally-closed (NC) configuration mentioned in section 4.3.5, which means that the end-stop breaks the circuit if pressed. Variable *A_LOCATION* saves home position as 0.

Listing 4.7 Homing sequence

```
1 def homeA() :  
2     pi.write(PINOUT.S_A["DIR"], 1)  
3     while readEndstopA() == 1:  
4         pi.write(PINOUT.S_A["STEP"], 1)  
5         delay.sleep(STEPPER_SPEED)  
6         pi.write(PINOUT.S_A["STEP"], 0)  
7         delay.sleep(STEPPER_SPEED)  
8     A_LOCATION = 0
```

Chapter 5

Results and Evaluation

In this chapter, test results are presented and discussed. Tests were carried out using both self-made tools and self-made setups. Tests performed include repeatability, accuracy, load capacity, deflection, and backlash tests. There exists a standard *ISO 9283* [28] for testing and evaluation of robotics, which document must be purchased for access. As a down payment, the testing environment and tools required to carry out ISO 9283 are expensive - ISO 9283 is discarded.

5.1 Results

All data points gathered from the repeatability test fall within the center of the target plate as shown in Figure 5.1 below.

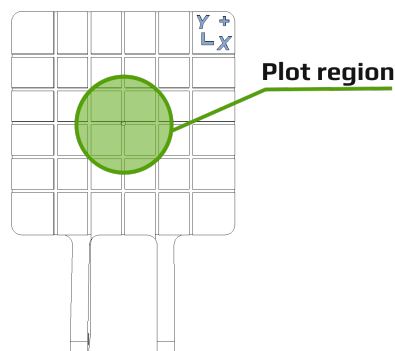


Figure 5.1: Region where data-points are located from both horizontal and vertical tests.

The results from the horizontal repeatability are shown in Figures 5.2, 5.3, 5.4, and 5.5 below.

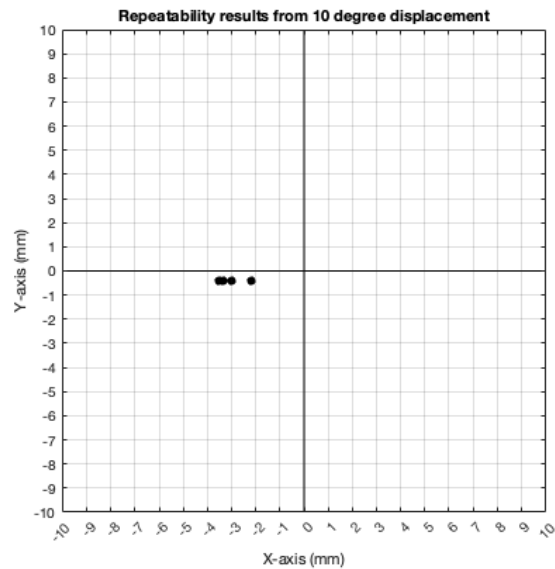


Figure 5.2: Results from 10° displacement.

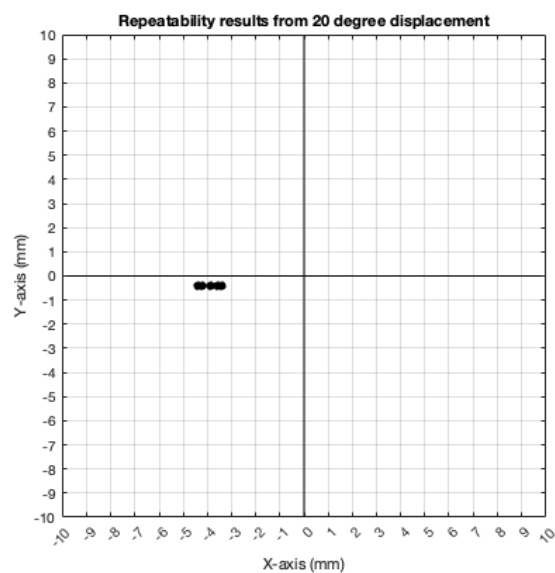


Figure 5.3: Results from 20° displacement.

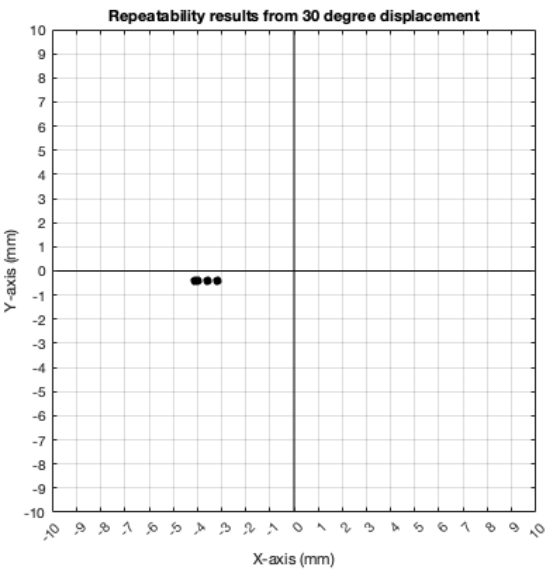


Figure 5.4: Results from 30° displacement.

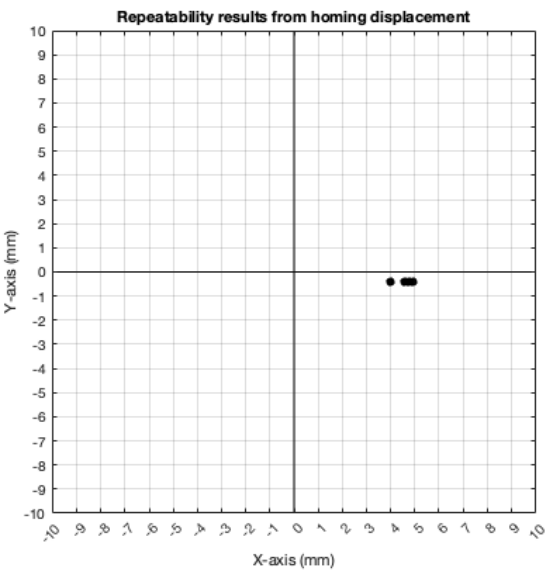


Figure 5.5: Results from homing displacement.

Results from the vertical repeatability test did not show any extreme deviations, and all fell within a radius of 0.5mm from the target-plate origin shown in Figure 5.6 below. All data points are located inside the center circle.

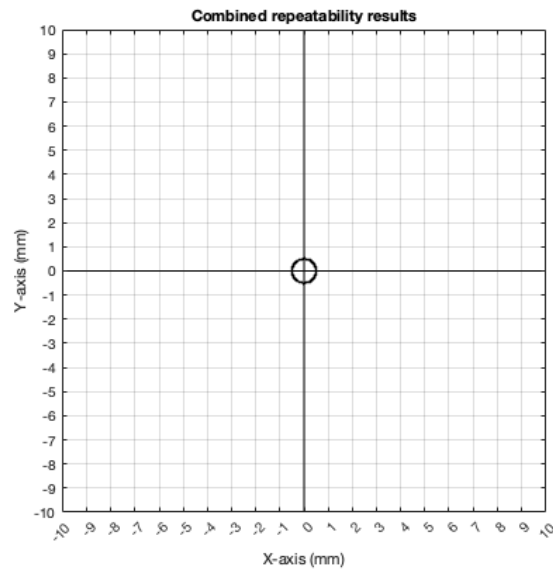


Figure 5.6: Vertical repeatability test results.

Maximum load capacity was found when attempting to move a 1m long aluminum-extrusion with a 450g weight. The extrusion was successfully moved from the folded-homing position to a fully extended position without losing steps. However, the outermost motor with a 5:1 reduction was not able to lift the extrusion **in isolation** when the arm was fully extended. At this point, further tests were stopped in fear of damaging the arm.

Deflection results obtained can be seen in Figure 5.7 below. The first four and last four data points are from two different measurements. The first measurement was carried out by putting batteries in a box mounted on the end-effector, and the second measurement was carried out by placing stepper motors and bearings on top of the end-effector.

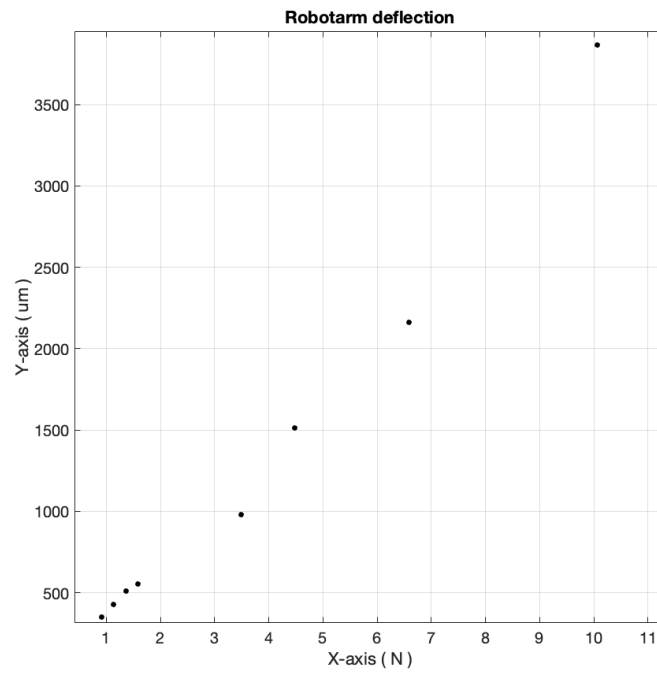


Figure 5.7: Deflection of the robotic-arm in μm under different loads.

Reach was determined with the use of forward-kinematics (FK) mathematical model derived in section 3.5.2. Results are shown in Figure 5.8 below.

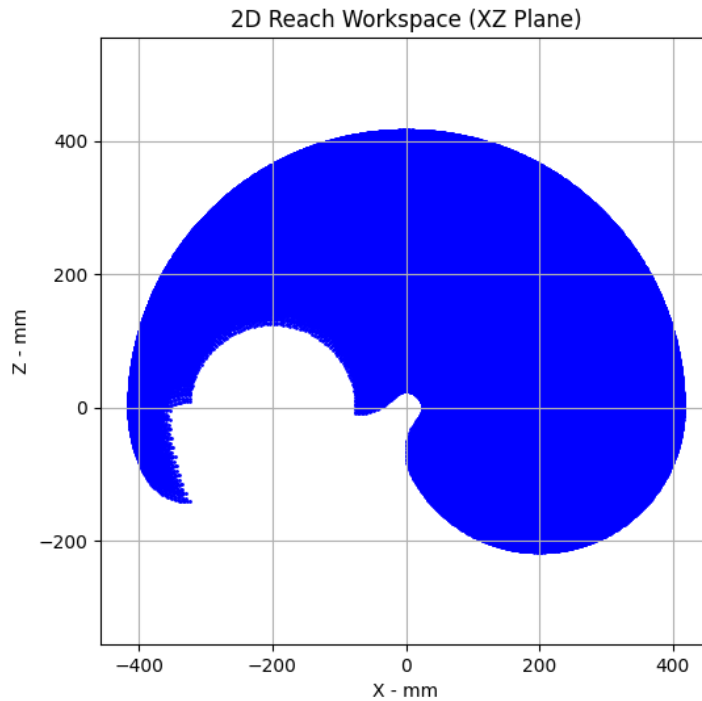


Figure 5.8: End-effector reach in the XZ-plane.

Accuracy tests were acquired through importing pictures of the robotic arm into CAD software and scaling them to match the real model. Once done, the measurements of the picture were taken. Results are compared to theoretical positions derived from the forward-kinematics model.

Table 5.1: Accuracy results real vs FK

Real X (mm)	Real Z (mm)	FK X (mm)	FK Z (mm)
-60	409	-66	400
34	246	36	260
198	254	200	266
358	162	341	207
399	65	400	66

Backlash was only calculated for the base plate where the robotic arm is mounted. As arm joints are constantly locked in one end of their min/max

backlash, they cannot be calculated by sending step-pulses and holding them still. The found backlash for the base-plate was $\approx 1.5^\circ$.

5.2 Evaluation

To fully understand the test results, the impact of backlash must be examined. Arm joints of the robotic arm have gearboxes with backlash tolerances $< 3^\circ$ [23]. Backlash impact is amplified by the arm joint length, and the maximum displacement error is calculated using:

$$e_{max} = l * \sin(\varphi) \quad (5.1)$$

Where φ is the backlash, l the length of the arm joint and e_{max} the maximum displacement-error. The longest arm joint of the robotic arm is 200mm, resulting in a maximum displacement error of $\approx 10.5\text{mm}$ for the joint. The combined backlash displacement of all joints is though to measure and was discarded.

The initial deviations from repeatability of the test are attributed to backlash. With a lack of acceleration and deceleration in movement, inertia caused by rotational movement introduces vibrations when motors are abruptly stopped. These vibrations cause unintended movement within the backlash range, displacing the arm from its target slightly.

As initial deviations from the origin of the target plate stay consistent throughout the test, the movement caused by vibrations is also consistent. Figure 5.9 below pictures how the repeatability error occurs. The white ball is the location where the test starts (needle centered on the target plate). Green ball is where the displacement movement start from and blue-ball is the final position of the arm - displaced slightly from the white position after rotational inertia induced vibrations. The grooves in which the ball can roll are the backlash range in which positional error is possible. The yellow rod in the back is the axis around which the joint rotates.

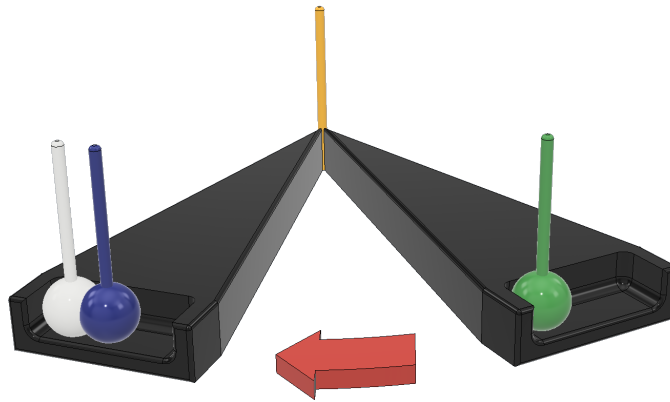
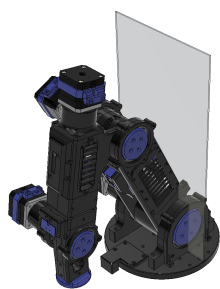
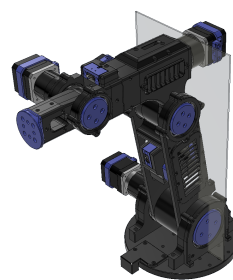


Figure 5.9: CAD render of backlash in rotational movement.

But why does the vertical test not produce the same positional error for the x-axis as the horizontal test? The answer is likely because of the mass displacement of the robotic arm. In the vertical test, mass is on both sides of the base plate and closer to the rotational point, whereas in the horizontal test, mass is located on one half of the base plate far out. The mass displacement is what amplifies inertia-induced vibrations in the horizontal test. Test positions used can be seen in Figure 5.10 below, which visualizes the mass displacement about the base plate of the arm.



(a) Horizontal repeatability-test setup



(b) Vertical repeatability-test setup

Figure 5.10: CAD render of test-positions about the center of the base-plate.

Load capacity results are not affected by backlash and are valid

measurements as the robotic arm in an extended position is locked into its backlash maximum due to gravity. The result shows that the furthest out motor was able to lift the extrusion from a homed position to a fully extended position. However, at the extended position, the motor in isolation struggled to lift the extrusion. The explanation for this discrepancy stems from the movement code, where the motors get their rotational speed distributed based on the slowest motor moving at a given moment. As the motor was moved in isolation, the speed value was too high, and enough torque was lost, resulting in skipped steps. Figure 5.11 below showcases the torque-speed relation of the stepper motor used.

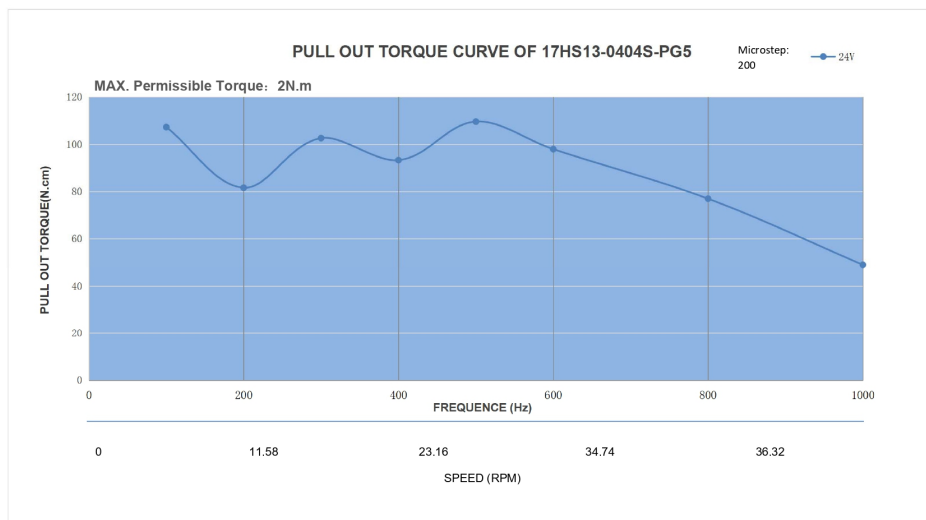


Figure 5.11: Data-sheet of speed vs torque for the stepper motor used.

The deflection of the robotic arm shows promising results and aligns with 3D-printed ABS plastic deflection tests reported in Fig. 8 of [29]. Most importantly, the data collected appears to be linear and could be offset if the load is known.

The Reach of the robot indicates a good design. The figure 5.12 below showcases a CAD rendering of the arm in different positions overlapped with figure 5.8, which validates the result and the forward-kinematics mathematical

model. Due to the base assembly of the robot, some points that are reachable in theory won't apply to the real model.

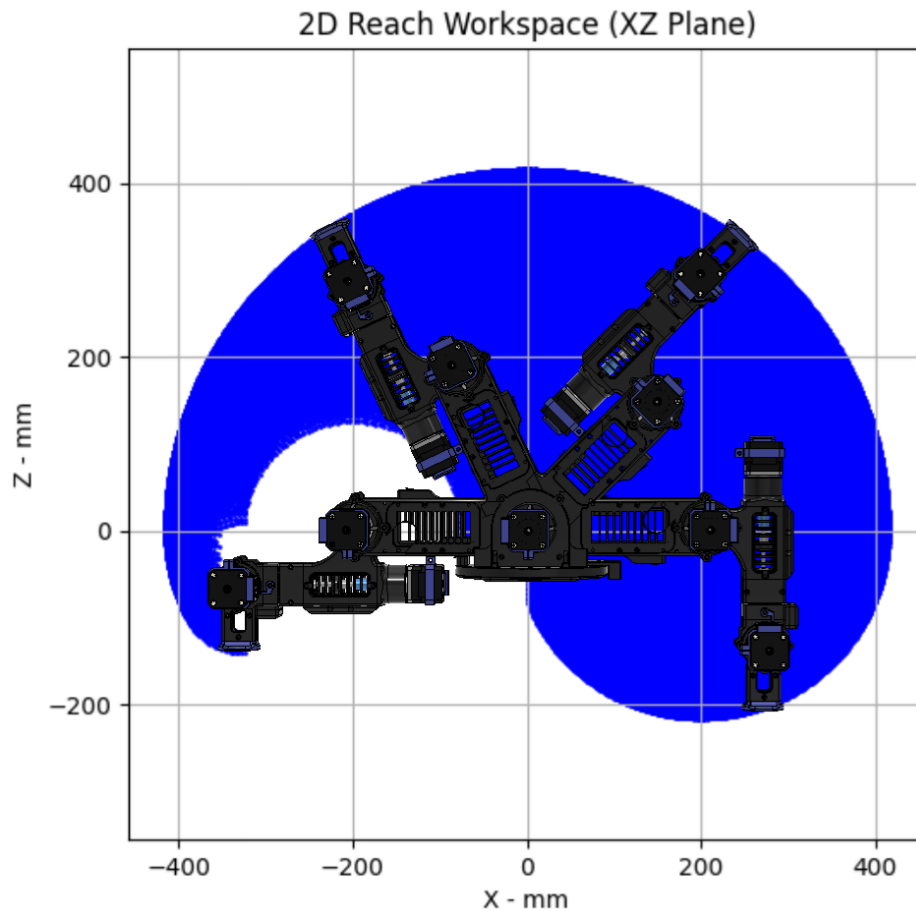


Figure 5.12: Reach of the robotic arm overlapped with simulated reach derived from forward-kinematics data-points.

The accuracy test results are rough and should be thought of as a hint at potential issues. The trend indicates somewhat stable results until all three motor angles trigger backlash in the same direction, seen in row four of table 5.1.

The conclusion from the evaluation is that all test setups were valid for testing a do-it-yourself robotic arm. Problems with backlash were identified through the accuracy and repeatability tests. Backlash causes positional errors that supersede all other inconsistencies of the robotic arm. The solution to the

problem involves gearboxes with better tolerances or the use of other types of gearboxes with low backlash. Load capacity, deflection, and reach results are promising for a do-it-yourself build. The backlash results for the base plate fall within the manufacturer's tolerance specification.

The robotic arm is not suitable for precision work until backlash is reduced significantly, and is better suited for educational purposes. Accuracy and backlash tests highlight difficulties of testing robotic arms without access to professional equipment.

Chapter 6

Conclusion and Future work

Achieving an operational robotic arm validates the design and development process undertaken. This achievement is amplified further as design, electronics, hardware, and software were integrated within 10 weeks from scratch. The functioning arm also validates the design of Tinkerer, which was developed outside the time frame of this project. Although not perfect, the robotic arm operates without noise or any critical issues. It does, however, have a backlash problem that must be solved to yield better results. Precision-planetary gearboxes can be found for $\approx 50\text{€}$ and have backlash of less than 0.5° , improving the backlash by a factor of 6. However, such an upgrade would cost $\approx 250\text{€}$ - and a noticeable backlash would persist, making it a poor option. A true solution to the backlash issues would be the use of cycloidal gearboxes for joints that carry a heavy load and belted gearboxes for joints that carry little load.

Some tests used for evaluation are problematic. Firstly, the repeatability test were hard to read and are prone to human error when measuring distance between center-point and needle-tip. The test setup itself is valid but a more accurate way of measuring deviations is required for data with higher resolution. Secondly, the accuracy test rely too much on good photos and can at best hint at compound backlash from multiple joints. For a system with very low backlash, the improvised accuracy test should be discarded completely. Thirdly, the backlash test can only measure backlash of rotational-joints, and not arm-joints. This is due to gravity pulling arm-joints automatically into its max backlash range, hindering the reversal test. Load capacity and deflection tests, which involve a high resolution dial-gauge yields good results and can be trusted. Not only are the tests simple to setup but they also involve expensive

equipment - a dial gauge. The conclusion to be made from the tests is that self-improvised tests work up to a certain point until expensive equipment is required. For testing of a self-made robotic arm, the improvised test methods can be used to check for blatant errors such as large backlash, but for finer errors, expensive equipment is required.

With the limited time frame and budget constraints, several features were excluded from the project. The robotic-arm build consumed most of the time reserved for the project, and little room was left for testing and improvements. Planned features for the future of the project can be read in the next section [6.1](#) of the chapter.

6.1 Future work

This section covers future work that is required for the robotic arm. With a limited development time frame, many features were discarded. As the robotic arm took much time to develop, points of improvement were discovered late in the time period of the project.

6.1.1 Monitoring GitHub page

To keep the project relevant in an open-source environment, further improvements to the robotic arm will be uploaded and documented. Feedback and questions will also be answered by the builders of the project.

6.1.2 Designing add-on end-effectors

The robotic arm in its current form has no way to lift objects. The end of the arm is fitted with heat-set inserts for custom add-on end-effectors. Development of a servo-driven gripper end-effector is planned for a start. Other fields to explore include pneumatic and electromagnetic end-effectors.

6.1.3 Tinkerer 1.0 optimizations

Although Tinkerer proved successful in this project, improvements can be made. A limited resource is the general-purpose input/output (GPIO) pins of the connected Raspberry Pi. To extend the number of GPIO outputs, shift registers can be used.

6.1.4 Nema-23 Stepper motors

Nema-23 stepper motors are the next size jump from the Nema-17 stepper motors used in the build. Higher motor torque allows for smaller gear reductions, which improves the speed at which joints of the arm can move. Nema-23 stepper motors come at a price hike, and so do the required drivers, which are necessary to handle the increased amperage requirements. Nema-23 stepper motors are especially beneficial for the lowest joint in the robotic arm.

Nema-23 stepper motors typically come with keyed shafts, which provide a stronger mechanical connection compared to D-cut shafts with set-screws for fastening. With a geared Nema-23 implementation, keyed flange-couplers must be purchased, and their geometry differs from flange-couplers used for D-cut shafts, resulting in the need for a new design of 3D-printed parts.

6.1.4.1 New stepper drivers

Nema-23 class of stepper motors typically uses large industrial stepper drivers. These stepper drivers take up significant space in the electronics compartment, and 3D-printed rigs for fastening would require a redesign to give space for new drivers. Increasing the size of copper traces on Tinkerer to meet increased amperage is not required, as these drivers are standalone.

6.1.5 Explore cycloidal-gearboxes

Cycloidal gearboxes, mentioned in section 4.2, are a low-backlash, high-torque solution which would be beneficial for the lower joints of the robotic arm. Its complicated design and tight tolerances for a 3D-printed assembly made it an impossible alternative given the project time frame. A good cycloidal gearbox would solve the backlash issue discovered while testing the robotic arm.

6.1.6 C-based software

Although the Python-based software runs, it's not optimal. C-based software runs quicker and will be a necessity when implementing more advanced movement in the future.

6.1.6.1 Acceleration & deceleration

Motors with gear-reductions and micro-stepping run sufficiently smooth and quiet for a demo, but could be improved further with acceleration and deceleration. Acceleration and deceleration were neglected. With a C-based software, space would exist to handle acceleration and deceleration, reducing vibrations caused by inertia when moving joints.

6.1.6.2 Inverse kinematics

Similar to the reason behind the neglect of acceleration and deceleration, the implementation of inverse kinematics was also neglected. Once features deemed more critical to the evolution of the project, like installing Nema-23 stepper motors on lower joints, inverse kinematics will be implemented.

References

- [1] VORON design. [Online]. Available: <https://vorondesign.com/> [Pages 2 and 14.]
- [2] Original CNC kitchen gewindeinsatz / threaded insert m4x4. [Online]. Available: <https://cnckitchen.store/products/made-for-voron-gewindeinsatz-threaded-insert-m3x5x4-100-stk-pcs> [Page 2.]
- [3] ruthex m3x5x4 thread insert - made for VORON - 100 pieces RX-m3x5x4 brass threaded bushings for 3d printed plastic parts. [Online]. Available: <https://www.ruthex.de/en/products/ruthex-gewindeinsatz-m3-100-stuck-made-for-voron-rx-m3x5x4-messing-gewindebuchsen-fur-3d-druck> [Page 2.]
- [4] MSc mechatronics | KTH | sweden. [Online]. Available: <https://www.kth.se/en/studies/master/mechatronics/msc-mechatronics-1.1262640> [Page 6.]
- [5] VORON2.4. [Online]. Available: <https://vorondesign.com/voron2.4> [Page 10.]
- [6] S. V, “naitzabez/voron-2.4-150mm,” original-date: 2021-09-28T17:57:07Z. [Online]. Available: <https://github.com/naitzabez/Voron-2.4-150mm> [Page 11.]
- [7] VSW_mods/MakerBeamEnclosure at main · naitzabez/VSW_mods. [Online]. Available: https://github.com/naitzabez/VSW_Mods/tree/main/MakerBeamEnclosure [Page 11.]
- [8] Micron/mods/sebbe at main · PrintersForAnts/micron. [Online]. Available: <https://github.com/PrintersForAnts/Micron/tree/main/Mods/sebbe> [Page 11.]

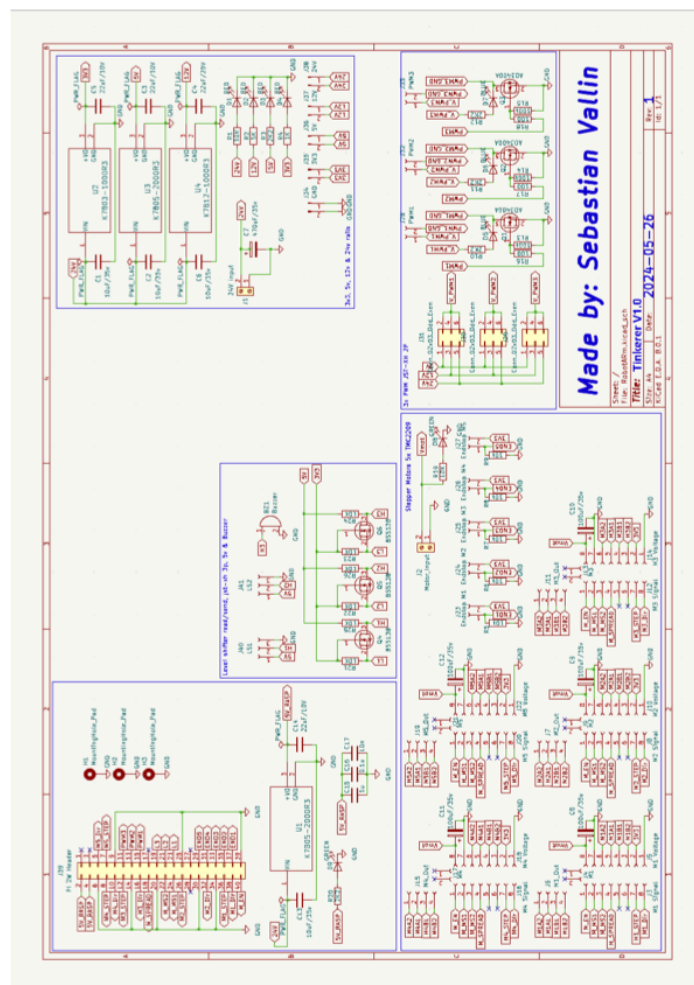
- [9] GOFA collaborative robot | go faster & further cobot - collaborative robots robot solutions & cobot technology (browse all ABB robots | robotics) | collaborative robots | ABB. [Online]. Available: <https://new.abb.com/products/robotics/robots/collaborative-robots/crb-15000> [Page 13.]
- [10] ABB IRB 120 robot | robots.com. [Online]. Available: <https://www.robots.com/industrial-robots/abb-irb-120> [Pages 13 and 23.]
- [11] Slicer - RepRap. [Online]. Available: <https://reprap.org/wiki/Slicer> [Page 15.]
- [12] M. Hamoud, A. B. Elshalakany, M. Gamil, and H. Mohamed, "Investigating the influence of 3d printing parameters on the mechanical characteristics of FDM fabricated (PLA/cu) composite material," vol. 134, no. 7, pp. 3769–3785. doi: 10.1007/s00170-024-14313-0. [Online]. Available: <https://doi.org/10.1007/s00170-024-14313-0> [Page 16.]
- [13] D. Syrlybayev, B. Zharylkassyn, A. Seisekulova, M. Akhmetov, A. Perveen, and D. Talamona, "Optimisation of strength properties of fdm printed parts—a critical review," *Polymers*, vol. 13, no. 10, 2021. doi: 10.3390/polym13101587. [Online]. Available: <https://www.mdpi.com/2073-4360/13/10/1587> [Page 16.]
- [14] ABS+ filament eSUN ABS+ 3d printer materials ABS plus. [Online]. Available: <https://www.esun3d.com/abs-pro-product/> [Page 17.]
- [15] A. Y. Davis, Q. Zhang, J. P. S. Wong, R. J. Weber, and M. S. Black, "Characterization of volatile organic compound emissions from consumer level material extrusion 3d printers," vol. 160, p. 106209. [Page 17.]
- [16] nevermore3d/nevermore_micro: Activated carbon filters. bad smells or fumes, or complaints thereof, should not keep you from being a maker! [Online]. Available: https://github.com/nevermore3d/Nevermore_Micro [Page 18.]
- [17] Backlash (engineering) - wikipedia. [Online]. Available: [https://en.wikipedia.org/wiki/Backlash_\(engineering\)](https://en.wikipedia.org/wiki/Backlash_(engineering)) [Page 18.]
- [18] D. Constantin, M. Lupoae, C. Baci, and B. Ilie, "Forward kinematic analysis of an industrial robot," 03 2015. [Page 22.]

- [19] Raspberry pi six-axis robotic arm - DFRobot. [Online]. Available: <https://www.dfrobot.com/product-2596.html> [Page 24.]
- [20] ReBeL® lightweight robot with cobot function. [Online]. Available: <https://www.igusab.se/info/build-or-buy-servicerobot-rebel> [Page 24.]
- [21] conk_b. How to 3d print key copy from photos. [Online]. Available: <https://www.instructables.com/How-To-3d-Print-Key-Copy-From-Photos/> [Page 27.]
- [22] E. Giovannitti, G. Squillero, and A. Tonda, *Virtual Measurement of the Backlash Gap in Industrial Manipulators*, 01 2020. [Page 29.]
- [23] Nema 17 stepper motor bipolar l=48mm w/ gear ratio 51:1 planetary gearbox - 17hs19-1684s-PG51 | StepperOnline. [Online]. Available: <https://www.omc-stepperonline.com/nema-17-stepper-motor-bipolar-1-48mm-w-gear-ratio-51-1-planetary-gearbox-17hs19-1684s-pg51> [Pages 34 and 59.]
- [24] Why choose a switch with NC/NO electrical function? [Online]. Available: <https://eshop.se.com/in/blog/post/why-choose-a-switch-with-nc-no-electrical-function.html> [Page 46.]
- [25] F. Zehra, M. Javed, D. Khan, and M. Pasha, “Comparative analysis of c++ and python in terms of memory and time.” doi: 10.20944/preprints202012.0516.v1 Publisher: Preprints. [Online]. Available: <https://doi.org/10.20944/preprints202012.0516.v1> [Page 48.]
- [26] pigpio library. [Online]. Available: <https://abyz.me.uk/rpi/pigpio/> [Page 49.]
- [27] “Daemon (computing),” page Version ID: 1288355106. [Online]. Available: [https://en.wikipedia.org/w/index.php?title=Daemon_\(computing\)&oldid=1288355106](https://en.wikipedia.org/w/index.php?title=Daemon_(computing)&oldid=1288355106) [Page 50.]
- [28] ISO 9283:1998 - manipulating industrial robots — performance criteria and related test methods. [Online]. Available: <https://www.iso.org/standard/22244.html> [Page 53.]
- [29] M. Somireddy and A. Czekanski, “Anisotropic material behavior of 3d printed composite structures – material extrusion additive manufacturing,” *Materials Design*, vol. 195, p. 108953, 07 2020. doi: 10.1016/j.matdes.2020.108953 [Page 61.]

Appendix A

Tinkerer 1.0 schematics and pictures

A.1 Schematics



A.2 Gallery

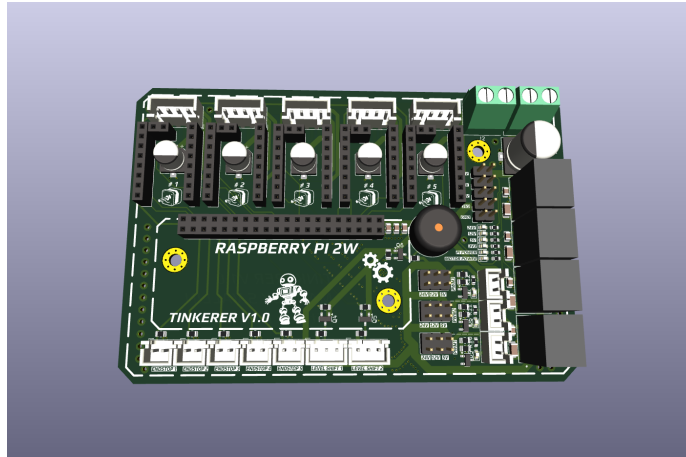


Figure A.1: Front view of Tinkerer 1.0

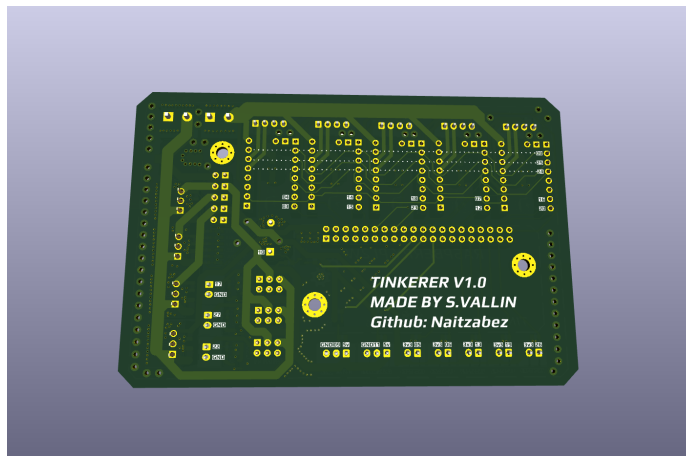


Figure A.2: Back view of Tinkerer 1.0

Appendix B

Cost, BOM and Miscellaneous data

Here you can read about the price of the build, the bill-of-materials (BOM), and miscellaneous data about the build.

B.1 Bill of materials and cost

Table [B.1](#) below summarizes the total cost of items required to build the project. Two price columns exist. **Price required** takes into account how many parts are used and reduces the cost from the original order accordingly. **Order price** is the total cost of orders placed, which sometimes contain more than necessary items.

Table B.1: BOM and project price breakdown

Quantity - Item	Price required (kr)	Order price (kr)
1 x Tinkerer 1.0 + SMD assembly	105	523
THT components (in stock)	250	N/A
1 x Meanwell LRS 150-24	330	330
1 x Generic 4010 fan (in stock)	50	N/A
1 x Raspberry PI Zero 2W (in stock)	219	N/A
1 x IEC320-C14 15A 230V inlet	91	181
5 x Micro-Switches	14	161
5 x TMC2226 Stepper Drivers in stock)	332	N/A
10m Braided PET-Sleeve 8mm	175	175
10m Braided PET-Sleeve 4mm	118	118
3kg eSUN ABS+	537	895
2020 Aluminum extrusions (in stock)	705	N/A
50 x M3 sliding T-nuts (in stock)	142	N/A
5 x Geared stepper motors (in stock)	1557	N/A
200 x M3 Brass heatset-inserts	270	270
1x Plexi-glass sheet 410mm x 160mm (in stock)	100	N/A
2 x Flange-coupler 5mm ID	42	125
1 x Flange-coupler 6mm ID	35	140
3 x Flange-coupler 8mm ID	105	140
1 x Brass axis-coupler 6mm-to-5mm	30	150
2 x Bearing 6812-2RS	240	240
2 x Bearing 6809-2RS	94	187
2 x Bearing 6807-2RS	64	127
2 x Bearing 6805-2RS	59	117
1 x Lazy-susan 95mm ²	110	110
1 x Stainless-Steel axis 5mm, 250mm length	58	116
10m Silicone wire (5 colors)	299	299
JST-XH 2.54mm assortment (in stock)	100	N/A
Molex-Microfit 3.0 assortment (in stock)	100	N/A
M3-screw assortment (in stock)	300	N/A
M4-screw assortment (in stock)	50	N/A
Total cost	6681	8344

B.2 Build data

In table B.2 below, miscellaneous data about the build is presented. Dimensions are measured from the CAD model, and so is the number of 3D-printed parts. Time spent printing is extracted from the history web interface used to control the Voron 3D-printer.

Table B.2: Miscellaneous build-data

Data	Value
3D-printed parts used (qty)	109
Time spent 3D-printing (hr)	111
ABS filament used (kg)	2+
Weight (kg)	8.5
Length (mm)	575
Width (mm)	328
Height (mm) (folded arm)	285
Height (mm) (fully extended arm)	589

Appendix C

Tools used

Here you can find all the tools used to construct the robotic arm. The tools are divided into physical tools and software tools, and are presented in lists with the name of the tool and its use.

C.1 Physical tools

The table [C.1](#) below lists the hand tools and equipment used in the assembly.

Table C.1: Tools used in the assembly

Tool name	Use
Voron 2.4 3D-printer	3D-printing
Digital calipers	Measurement
Set of allen-keys	Tightening bolts
Screwdrivers	Tightening bolts
Engineer PA-06	Wire stripping
Engineer PA-09	Crimping end-fittings
Engineer NS-06	Cutting
Ferrule-press	Crimping ferrules
Miniware TS101	Soldering
Fluke 175	Multimeter
Power drill	Drilling holes for blind joints
Cigarette-lighter	Shrink-tube heating

C.2 Development software

Table C.2 below lists the software and what it was used for in the project.

Table C.2: Software used in the project

Software name	Use
Fusion 360	Design, 3D-modeling and simulation
PrusaSlicer	Slicing 3D-models for 3D-printing
Visual Studio Code	Programming environment
Raspberry PI Imager	Flashing OS to Raspberry

Appendix D

Code

Here you find the code used in the project.

D.1 Pinout

```
1  #Pinout for Tinkerer PCB board
2  S_EN = {
3      "EN" : 21
4  }
5  S_MS1 = {
6      "MS" : 8
7  }
8  S_MS2 = {
9      "MS" : 25
10 }
11 S_SP = {
12     "SPREAD" : 24
13 }
14 S_A = {
15     "DIR" : 20,
16     "STEP" : 16
17 }
18 S_B = {
19     "DIR" : 12,
20     "STEP" : 7
21 }
22 S_C = {
```

```
23     "DIR":23,  
24     "STEP":18  
25 }  
26 S_D = {  
27     "DIR":15,  
28     "STEP":14  
29 }  
30 S_E = {  
31     "DIR":3,  
32     "STEP":4  
33 }  
34 E_A = {  
35     "END":26  
36 }  
37 E_B = {  
38     "END":19  
39 }  
40 E_C = {  
41     "END":13  
42 }  
43 E_D = {  
44     "END":6  
45 }  
46 E_E = {  
47     "END":5  
48 }  
49 PWM1 = {  
50     "PWM":3  
51 }  
52 PWM2 = {  
53     "PWM":2  
54 }  
55 PWM3 = {  
56     "PWM":0  
57 }
```

D.2 Main-program

```

1  #Maps GPIO and inits
2  import pinout as PINOUT
3  import time as delay
4  import math
5  import pigpio
6  import threading
7  import random
8
9  pi = pigpio.pi()
10 #Stepper Vars
11 STEPPER_SPEED = 0.0001
12 SPEED = 0.00003
13 MSTOTAL = 8
14 STEPCOUNT = 200
15
16 A_LOCATION = 0
17 B_LOCATION = 0
18 C_LOCATION = 0
19 D_LOCATION = 0
20 E_LOCATION = 0
21
22 def pinSetup():
23     delay.sleep(0.2)
24     #Mapping for all steppers:
25     pi.set_mode(PINOUT.S_MS1["MS"], pigpio.OUTPUT)
26     pi.set_mode(PINOUT.S_MS2["MS"], pigpio.OUTPUT)
27     pi.set_mode(PINOUT.S_EN["EN"], pigpio.OUTPUT)
28
29     ↪ pi.set_mode(PINOUT.S_SP["SPREAD"], pigpio.OUTPUT)
30     #Stepper A mapping:
31     pi.set_mode(PINOUT.S_A["DIR"], pigpio.OUTPUT)
32     pi.set_mode(PINOUT.S_A["STEP"], pigpio.OUTPUT)
33     #Stepper B mapping:
34     pi.set_mode(PINOUT.S_B["DIR"], pigpio.OUTPUT)
35     pi.set_mode(PINOUT.S_B["STEP"], pigpio.OUTPUT)
36     #Stepper C mapping:
37     pi.set_mode(PINOUT.S_C["DIR"], pigpio.OUTPUT)

```

```

37 pi.set_mode(PINOUT.S_C["STEP"], pigpio.OUTPUT)
38 #Stepper D mapping:
39 pi.set_mode(PINOUT.S_D["DIR"], pigpio.OUTPUT)
40 pi.set_mode(PINOUT.S_D["STEP"], pigpio.OUTPUT)
41 #Stepper E mapping:
42 pi.set_mode(PINOUT.S_E["DIR"], pigpio.OUTPUT)
43 pi.set_mode(PINOUT.S_E["STEP"], pigpio.OUTPUT)
44
45 #ENDSTOP_A
46 pi.set_mode(PINOUT.E_A["END"], pigpio.INPUT)
47 #ENDSTOP_B
48 pi.set_mode(PINOUT.E_B["END"], pigpio.INPUT)
49 #ENDSTOP_C
50 pi.set_mode(PINOUT.E_C["END"], pigpio.INPUT)
51 #ENDSTOP_D
52 pi.set_mode(PINOUT.E_D["END"], pigpio.INPUT)
53 #ENDSTOP_E
54 pi.set_mode(PINOUT.E_E["END"], pigpio.INPUT)
55
56 #PWM
57 pi.set_mode(PINOUT.PWM1["PWM"], pigpio.OUTPUT)
58 pi.set_mode(PINOUT.PWM2["PWM"], pigpio.OUTPUT)
59 #pi.set_mode(PINOUT.PWM3["PWM"], pigpio.OUTPUT)
60
61 def default_start():
62     delay.sleep(0.2)
63     #Stepper General Setup
64     pi.write(PINOUT.S_MS1["MS"], 0)
65     pi.write(PINOUT.S_MS2["MS"], 0)
66     pi.write(PINOUT.S_SP["SPREAD"], 1)
67     pi.write(PINOUT.S_EN["EN"], 0)
68
69     #Stepper A setup
70     pi.write(PINOUT.S_A["DIR"], 0)
71     pi.write(PINOUT.S_A["STEP"], 0)
72     #Stepper B setup
73     pi.write(PINOUT.S_B["DIR"], 0)
74     pi.write(PINOUT.S_B["STEP"], 0)
75     #Stepper C setup

```

```

76     pi.write(PINOUT.S_C["DIR"], 0)
77     pi.write(PINOUT.S_C["STEP"], 0)
78     #Stepper D setup
79     pi.write(PINOUT.S_D["DIR"], 0)
80     pi.write(PINOUT.S_D["STEP"], 0)
81     #Stepper E setup
82     pi.write(PINOUT.S_E["DIR"], 0)
83     pi.write(PINOUT.S_E["STEP"], 0)
84     #Location vars
85
86
87     def readEndstopA():
88         return pi.read(PINOUT.E_A["END"])
89
90     def homeA():
91         pi.write(PINOUT.S_A["DIR"], 1)
92         print("Homing A....")
93         while readEndstopA() == 1:
94             pi.write(PINOUT.S_A["STEP"], 1)
95             delay.sleep(STEPPER_SPEED)
96             pi.write(PINOUT.S_A["STEP"], 0)
97             delay.sleep(STEPPER_SPEED)
98             A_LOCATION = 0
99         print("Homing A complete")
100
101     def readEndstopB():
102         return pi.read(PINOUT.E_B["END"])
103
104     def homeB():
105         pi.write(PINOUT.S_B["DIR"], 0)
106         print("Homing B....")
107         while readEndstopB() == 1:
108             pi.write(PINOUT.S_B["STEP"], 1)
109             delay.sleep(STEPPER_SPEED)
110             pi.write(PINOUT.S_B["STEP"], 0)
111             delay.sleep(STEPPER_SPEED)
112             B_LOCATION = 0
113         print("Homing B complete")
114

```



```

115 def readEndstopC():
116     return pi.read(PINOUT.E_C["END"])
117
118 def homeC():
119     pi.write(PINOUT.S_C["DIR"], 1)
120     print("Homing C....")
121     while readEndstopC() == 1:
122         pi.write(PINOUT.S_C["STEP"], 1)
123         delay.sleep(STEPPER_SPEED)
124         pi.write(PINOUT.S_C["STEP"], 0)
125         delay.sleep(STEPPER_SPEED)
126         C_LOCATION = 0
127     print("Homing A complete")
128
129 def readEndstopD():
130     return pi.read(PINOUT.E_D["END"])
131
132 def homeD():
133     pi.write(PINOUT.S_D["DIR"], GPIO.HIGH)
134     print("Homing D....")
135     while readEndstopD() == 1:
136         pi.write(PINOUT.S_D["STEP"], 1)
137         delay.sleep(STEPPER_SPEED)
138         pi.write(PINOUT.S_D["STEP"], 0)
139         delay.sleep(STEPPER_SPEED)
140         D_LOCATION = 0
141     print("Homing D complete")
142
143 def readEndstopE():
144     return pi.read(PINOUT.E_E["END"])
145
146 def homeE():
147     pi.write(PINOUT.S_E["DIR"], 0)
148     print("Homing E....")
149     while readEndstopE() == 1:
150         pi.write(PINOUT.S_E["STEP"], 1)
151         delay.sleep(STEPPER_SPEED)
152         pi.write(PINOUT.S_E["STEP"], 0)
153         delay.sleep(STEPPER_SPEED)

```

```

154         E_LOCATION = 0
155     print("Homing E complete")
156
157     def program():
158         while True:
159             print("Welcome, enter 5 angles with spaces
160                   ↳ for the robot arm.")
161             print("Angles must be in motor order A B C
162                   ↳ D E")
163             print("Press x to exit.")
164             print("Type your input below:")
165             user_input = input("")
166             if user_input == "x":
167                 print("Exiting.")
168                 exit()
169             while True:
170                 numbertest = False
171                 angles = user_input.split()
172                 numbers = [0] * 5
173                 if len(angles) != 5:
174                     print("Format error, please
175                           ↳ retry.")
176                     break
177                 for i in range(5):
178                     try:
179                         numbers[i] = float(angles[i])
180                     except ValueError:
181                         print("Format error, not
182                               ↳ numbers.")
183                         numbertest = True
184                         break
185                 if numbertest:
186                     break
187                 motorTimedCalc(numbers)
188                 break
189
190     def runA(steps, speed):
191         for i in range(steps):
192             pi.write(PINOUT.S_A["STEP"], 1)

```

```

189         delay.sleep(speed)
190         pi.write(PINOUT.S_A["STEP"], 0)
191         delay.sleep(speed)
192
193     def runB(steps, speed):
194         for i in range(steps):
195             pi.write(PINOUT.S_B["STEP"], 1)
196             delay.sleep(speed)
197             pi.write(PINOUT.S_B["STEP"], 0)
198             delay.sleep(speed)
199
200     def runC(steps, speed):
201         for i in range(steps):
202             pi.write(PINOUT.S_C["STEP"], 1)
203             delay.sleep(speed)
204             pi.write(PINOUT.S_C["STEP"], 0)
205             delay.sleep(speed)
206
207     def runD(steps, speed):
208         for i in range(steps):
209             pi.write(PINOUT.S_D["STEP"], 1)
210             delay.sleep(speed)
211             pi.write(PINOUT.S_D["STEP"], 0)
212             delay.sleep(speed)
213
214     def runE(steps, speed):
215         for i in range(steps):
216             pi.write(PINOUT.S_E["STEP"], 1)
217             delay.sleep(speed)
218             pi.write(PINOUT.S_E["STEP"], 0)
219             delay.sleep(speed)
220
221     def motorTimedCalc(deg):
222         degArray = [0] * 5
223         stepArray = [0] * 5
224         speedArray = [0] * 5
225         global A_LOCATION, B_LOCATION, C_LOCATION,
226             ↪ D_LOCATION, E_LOCATION
227         if 0 <= deg[0] <= 180:

```

```

227         if A_LOCATION < deg[0]:
228             pi.write(PINOUT.S_A["DIR"], 0)
229             degArray[0] = deg[0]-A_LOCATION
230         else:
231             pi.write(PINOUT.S_A["DIR"], 1)
232             degArray[0] = A_LOCATION-deg[0]
233         A_LOCATION = deg[0]
234         print(f"Degrees to move: {degArray[0]}")
235         ratioA = degArray[0]/360
236         stepArray[0] = math.floor(13.73*8*200 *
237             ↪ ratioA)
238
239     if 0 <= deg[1] <= 180:
240         if B_LOCATION < deg[1]:
241             pi.write(PINOUT.S_B["DIR"], 1)
242             degArray[1] = deg[1]-B_LOCATION
243         else:
244             pi.write(PINOUT.S_B["DIR"], 0)
245             degArray[1] = B_LOCATION-deg[1]
246         B_LOCATION = deg[1]
247         print(f"Degrees to move: {degArray[1]}")
248         ratioB = degArray[1]/360
249         stepArray[1] = math.floor(50.9*8*200 *
250             ↪ ratioB)
251
252     if 0 <= deg[2] <= 180:
253         if C_LOCATION < deg[2]:
254             pi.write(PINOUT.S_C["DIR"], 0)
255             degArray[2] = deg[2]-C_LOCATION
256         else:
257             pi.write(PINOUT.S_C["DIR"], 1)
258             degArray[2] = C_LOCATION-deg[2]
259         C_LOCATION = deg[2]
260         print(f"Degrees to move: {degArray[2]}")
261         ratioC = degArray[2]/360
262         stepArray[2] = math.floor(26.85*8*200 *
263             ↪ ratioC)
264
265     if 0 <= deg[3] <= 90:

```

```

263     if D_LOCATION < deg[3]:
264         pi.write(PINOUT.S_D["DIR"], 0)
265         degArray[3] = deg[3]-D_LOCATION
266     else:
267         pi.write(PINOUT.S_D["DIR"], 1)
268         degArray[3] = D_LOCATION-deg[3]
269     D_LOCATION = deg[3]
270     print(f"Degrees to move: {degArray[3]}")
271     ratioD = degArray[3]/360
272     stepArray[3] = math.floor(26.85*8*200 *
        ↪ ratioD)
273
274     if 0 <= deg[4] <= 180:
275         if E_LOCATION < deg[4]:
276             pi.write(PINOUT.S_E["DIR"], 1)
277             degArray[4] = deg[4]-E_LOCATION
278         else:
279             pi.write(PINOUT.S_E["DIR"], 0)
280             degArray[4] = E_LOCATION-deg[4]
281         E_LOCATION = deg[4]
282         print(f"Degrees to move: {degArray[4]}")
283         ratioE = degArray[4]/360
284         stepArray[4] = math.floor(5.18*8*200 *
        ↪ ratioE)
285
286     mostStep = max(stepArray)
287     totalTime = SPEED*mostStep
288
289     for i in range(5):
290         if(stepArray[i] == 0):
291             continue
292         speedArray[i] = totalTime / (stepArray[i])
293
294     print("Calculations done, running motors..")
295     runMotors(stepArray, speedArray)
296
297     def runMotors(stepArray, speedArray):
298         MOTOR_A = threading.Thread(target=runA,
        ↪ args=(stepArray[0], speedArray[0]))

```

```

299     MOTOR_B = threading.Thread(target=runB,
    ↪     args=(stepArray[1], speedArray[1]))
300     MOTOR_C = threading.Thread(target=runC,
    ↪     args=(stepArray[2], speedArray[2]))
301     MOTOR_D = threading.Thread(target=runD,
    ↪     args=(stepArray[3], speedArray[3]))
302     MOTOR_E = threading.Thread(target=runE,
    ↪     args=(stepArray[4], speedArray[4]))
303
304     MOTOR_A.start()
305     MOTOR_B.start()
306     MOTOR_C.start()
307     MOTOR_D.start()
308     MOTOR_E.start()
309
310     MOTOR_A.join()
311     MOTOR_B.join()
312     MOTOR_C.join()
313     MOTOR_D.join()
314     MOTOR_E.join()
315     print("Movement completed!")
316
317     def activateFan():
318         pi.write(PINOUT.PWM1["PWM"], 0)
319         pi.write(PINOUT.PWM2["PWM"], 0)
320         print("Fan has been activated.")
321
322     def run():
323         if not pi.connected:
324             print("Unable to connect to pigpio daemon")
325             exit()
326         #setup()
327         print("Pin setup configuration enabled")
328         pinSetup()
329         print("Pin setup configured successfully")
330         default_start()
331         print("Default values enabled on pins")
332         delay.sleep(0.2)
333         activateFan()

```

```

334     print("Ready to rock!")
335     homeA()
336     homeE()
337     homeD()
338     homeB()
339     homeC()
340     program()
341 run()

```

D.3 Forward Kinematics

```

1  import numpy as np
2
3  def dh_matrix(theta, alpha, d, a):
4      ct, st = np.cos(theta), np.sin(theta)
5      ca, sa = np.cos(alpha), np.sin(alpha)
6      return np.array([
7          [ct, -st*ca, st*sa, a*ct],
8          [st, ct*ca, -ct*sa, a*st],
9          [0, sa, ca, d],
10         [0, 0, 0, 1]
11     ])
12
13 deg = np.deg2rad
14 input = deg([0, 0, 0, 0, 0])
15 theta = input + deg([0, 0, -90, 0, -180])
16 alpha = deg([-90, 180, 90, 90, -90])
17 d = [0, 0, 0, -125, 0]
18 a = [0, -200, -66, 0, 75]
19
20 T = np.eye(4)
21 for i in range(5):
22     T = T @ dh_matrix(theta[i], alpha[i], d[i],
23                       ↪ a[i])
24
25 position = np.round(T[:3, 3], 3)
26 print(f"End-effector position: X: {position[0]}, Y:
27       ↪ {position[1]}, Z: {position[2]}")

```

Appendix E

Accuracy Measurements

Here you can find pictures used to determine the accuracy of the real robotic arm. The measurements were carried out in the XZ-plane.



Figure E.1: End-effector position at: 0° , 90° , 90° , 0° , 90°

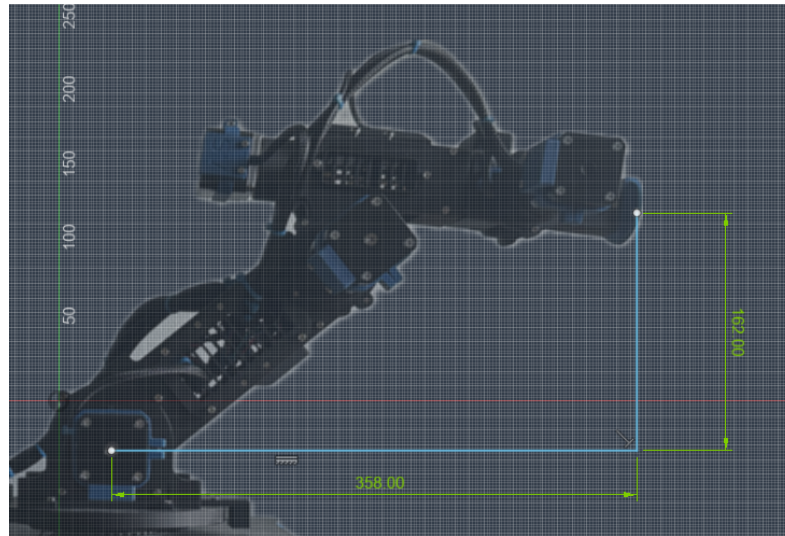


Figure E.2: End-effector position at: 0° , 135° , 135° , 0° , 90° .

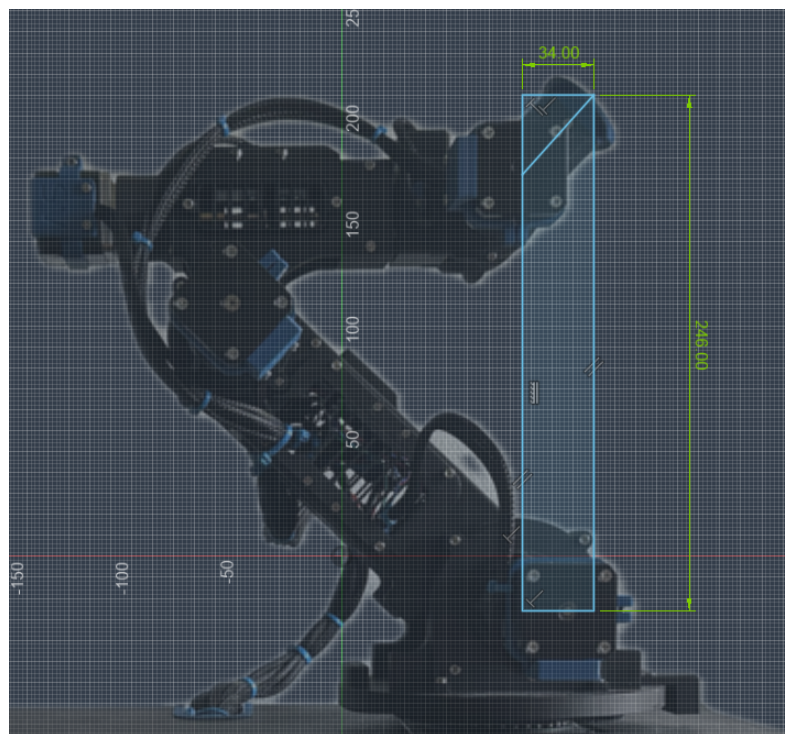


Figure E.3: End-effector position at: 0° , 45° , 45° , 0° , 45° .



Figure E.4: End-effector position at: $0^\circ, 90^\circ, 180^\circ, 0^\circ, 90^\circ$.

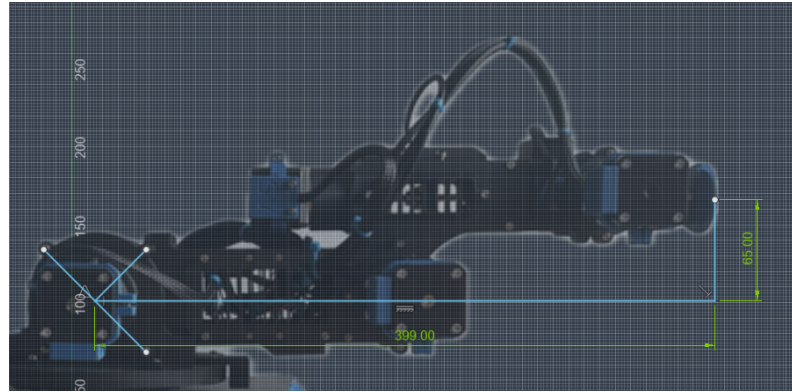


Figure E.5: End-effector position at: $0^\circ, 180^\circ, 180^\circ, 0^\circ, 90^\circ$.

€€€€ For DIVA €€€€

```
{
  "Author1": { "Last name": "Vallin",
    "First name": "Sebastian",
    "Local User Id": "sebval",
    "E-mail": "sebval@kth.se",
    "organisation": { "L1": "School of Electrical Engineering and Computer Science",
    }
  },
  "Cycle": "1",
  "Course code": "IL142X",
  "Credits": "15.0",
  "Degree1": { "Educational program": "Degree Programme in Electronics and Computer Engineering"
    , "programcode": "TIEDB"
    , "Degree": "Bachelors degree"
    , "subjectArea": "Technology"
  },
  "Title": {
    "Main title": "Open Source Five-axis Robotic-arm",
    "Subtitle": "Design, Construction and Evaluation",
    "Language": "eng"
  },
  "Alternative title": {
    "Main title": "Fem-axlad Robot-arm med öppen källkod",
    "Subtitle": "Design, Konstruktion och Utvärdering ",
    "Language": "swe"
  },
  },
  "Supervisor1": { "Last name": "Zetterling",
    "First name": "Carl-Mikael",
    "Local User Id": "bellmon",
    "E-mail": "bellman@kth.se",
    "organisation": { "L1": "School of Electrical Engineering and Computer Science",
    "L2": "Electronics & Embedded Systems" }
  },
  "Examiner1": { "Last name": "Rusu",
    "First name": "Ana",
    "Local User Id": "arusu",
    "E-mail": "arusu@kth.se",
    "organisation": { "L1": "School of Electrical Engineering and Computer Science",
    "L2": "Electronics & Embedded Systems" }
  },
  "National Subject Categories": "10201, 10206",
  "Other information": { "Year": "2025", "Number of pages": "1,97" },
  "Copyrightleft": "copyright",
  "Series": { "Title of series": "TRITA – EECS-EX", "No. in series": "2024:0000" },
  "Opponents": { "Name": "I. Hossain" },
  "Presentation": { "Date": "2025-06-12 13:15"
    , "Language": "eng"
    , "Room": "EECS Amiga"
    , "Address": "Isafjordsgatan 22 (Kistagången 16)"
    , "City": "Stockholm"
    , "Number of lang instances": "2",
    "Abstract[eng ]": €€€€
  },
  "Abstract[eng ]": €€€€
```

Denna avhandling utforskar upprepbarhet och precision som är möjlig med en egen-byggd fem-axlad robot-arm. Robot-armen utvecklades under begränsad budget med standardiserad hårdvara och 3D-utskrivna delar. Projektet har öppen källkod och all design har skett med hänsyn till tillgänglighet och återskapande. Robot-armen bygger på en tidigare-utvecklad prototyp som har förbättrats och utvärderats. Målet med robot-armen är att erbjuda en dokumenterad, testad robot-arm med öppen-källkod för att accelerera dess fortsatta utveckling. Utvärdering utfördes med själv-utvecklade test-metoder för att säkerställa precision, upprepbarhet, deflektion, glapp, arbetsområde samt belastningskapacitet. Erhållna resultat visar blandad prestanda med möjligheter till förbättring. Alla filer för 3D-modeller, kretskorts samt mjukvara finns tillgängligt i projektets GitHub arkiv.

```
€€€€,
"Keywords[eng ]": €€€€
3D-printing, Design process, Computer-aided design, Joints, Stepper motor, Stepper driver, Micro-stepping, Gear reduction, Torque, Speed,
3D-model, Assembly, Open-source, End-stop, Accuracy, Repeatability, Load capacity, Deflection, Backlash, Reach, Mathematical model,
Forward kinematics, Inverse kinematics €€€€,
"Abstract[swe ]": €€€€
€€€€,
"Keywords[swe ]": €€€€
3D-Printing, Designprocess, Dator hjälpt deisgn, Leder, Stegmotor, Stegmotor drivare, Microsteg, Nedväxling, Vridmoment, Hastighet,
3D-Model, Montage, Öppen-källkod, Ändstopp, Precision, Upprepbarhet, Glapp, Deflektion, Arbetsområde och Belastningskapacitet,
Matematisk modell, Framåt kinematik, Inverterad kinematik €€€€,
}
```

acronyms

```
%%% Local Variables:
%%% mode: latex
%%% TeX-master: t
%%% End:
% The following command is used with glossaries-extra
\setabbreviationstyle[acronym]{long-short}
% The form of the entries in this file is \newacronym{label}{acronym}{phrase}
%                                     or \newacronym[options]{label}{acronym}{phrase}
% see "User Manual for glossaries.sty" for the details about the options, one example is shown below
% note the specification of the long form plural in the line below
\newacronym{CAD}{CAD}{Computer Aided Design}
\newacronym{DIY}{DIY}{Do It Yourself}
\newacronym{DOF}{DOF}{Degrees Of Freedom}
\newacronym{5-DOF}{5-DOF}{Five Degrees Of Freedom}
\newacronym{PCB}{PCB}{Printed Circuit Board}
\newacronym{GPIO}{GPIO}{General-Purpose Input/Output}
\newacronym{PID}{PID}{Proportional-Integral-Derivative}
\newacronym{PWM}{PWM}{Pulse Width Modulation}
\newacronym{NO}{NO}{Normally Open}
\newacronym{NC}{NC}{Normally Closed}
\newacronym{FK}{FK}{Forward Kinematics}
\newacronym{IK}{IK}{Inverse Kinematics}
```